

27 — Security and Incident Response Plan

Edición PDF con identidad visual unificada de VRM Games. El contenido procede íntegramente del archivo Markdown original.

PROYECTO

Cartridge & Cloud

ESTADO

Documento de la baseline RC1

DOCUMENTO

27 / 27_Security_and_Incident_Response_Plan.md

AUTORÍA

VRM Games / Blas Luis Rocha González

VERSIÓN

RC1

FECHA DOCUMENTAL

2026-07-02

Control y metadatos del documento

Archivo fuente: 27_Security_and_Incident_Response_Plan.md

SHA-256 del Markdown: f925bfbf3df2000323c12a4486480d06c8ab8181684e5922168b8bdf89d05f92

Tamaño del Markdown: 422,182 bytes

Conversión: representación visual completa; el Markdown original mantiene la autoridad sobre el texto fuente.

Front matter original

El archivo no contiene front matter YAML.

Índice

27 — Security and Incident Response Plan

0. Propósito, alcance y autoridad

1. Genealogía histórica de seguridad

2. Estado de seguridad actual

3. Principios Secure by Design

4. Gobernanza y responsabilidad

5. Modelo de amenazas

6. Activos que proteger

7. Superficies de ataque

8. Límites de confianza

9. Clasificación de información

10. Código fuente

11. Repositorios y ramas

12. Credenciales y secretos

13. Steamworks y cuentas de partner

14. Unity ID y servicios

15. GitHub y dispositivos

16. Estaciones de trabajo

17. Blender y herramientas DCC

18. Paquetes y dependencias

19. Supply chain de Unity Package Manager

20. SBOM

21. Licencias y seguridad

22. Artefactos de build

23. Integridad y checksums

24. Firmas y procedencia

25. Backups del repositorio

26. Backups documentales

27. Backups de herramientas y fuentes

28. Recuperación ante pérdida total

29. Control de acceso

30. Principio de mínimo privilegio

31. MFA

32. Altas, bajas y cambios de acceso

33. Roles de colaborador

34. Acceso de proveedores

35. Cuentas compartidas

36. Password manager

37. Rotación de secretos

38. Secret scanning

39. Variables de entorno

40. Archivos VDF y SteamPipe

41. API keys futuras

42. Configuración segura por defecto

43. Hardening de Windows de desarrollo

44. Actualizaciones del sistema

45. Antimalware y falsos positivos

46. Cifrado de dispositivo

47. Bloqueo y sesión

48. Medios extraíbles

49. Trabajo remoto

50. Wi-Fi y red

51. Descargas y fuentes confiables

52. Macros y documentos

53. Contenido generado y archivos externos

54. Importación de Unity packages

- | | |
|---|-------------------------------------|
| 55. Editor scripts y ejecución automática | 87. Autorización futura |
| 56. Build scripts | 88. Steam ID futuro |
| 57. CI/CD futuro | 89. Anti-cheat y fraude |
| 58. Protección de main | 90. Manipulación de saves |
| 59. Code review | 91. Integridad de economía offline |
| 60. Change review de ProjectSettings | 92. Modding y Workshop futuros |
| 61. Change review de Packages | 93. Plugins nativos |
| 62. Dependency pinning | 94. Native DLLs |
| 63. Evaluación de actualizaciones | 95. Signing de ejecutables |
| 64. Vulnerabilidades conocidas | 96. Reputación y SmartScreen |
| 65. CVE y advisories | 97. Installer y distribución |
| 66. Política de parcheo | 98. Steam depots y branches |
| 67. Ventanas de mantenimiento | 99. Rollback seguro |
| 68. Excepciones de seguridad | 100. Previous stable |
| 69. Aceptación de riesgo | 101. Release candidate |
| 70. Secure coding C# | 102. Reproducibilidad |
| 71. Validación de entrada | 103. Hash manifest |
| 72. Paths y traversal | 104. THIRD_PARTY_NOTICES |
| 73. Deserialización y JSON | 105. Security test plan |
| 74. Archivos temporales | 106. Static analysis |
| 75. Atomicidad de saves | 107. Dependency scanning |
| 76. Corrupción y recuperación | 108. Secret scanning automatizado |
| 77. Integer overflow y economía | 109. Malware scanning de artefactos |
| 78. Idempotencia | 110. Fuzzing selectivo |
| 79. Excepciones y fail-safe | 111. Tests de paths y archivos |
| 80. Logging seguro | 112. Tests de corrupción |
| 81. Redacción de logs | 113. Tests de rollback |
| 82. Dumps y datos sensibles | 114. Tests de permisos |
| 83. Network code futuro | 115. Tests en usuario estándar |
| 84. TLS y certificados | 116. Clean machine test |
| 85. Backend futuro | 117. Proton/Deck security review |
| 86. Autenticación futura | 118. Incident taxonomy |

119. Severidades SEC-0 a SEC-4
120. Detección
121. Canales de reporte
122. Vulnerability disclosure
123. Recepción responsable de reportes
124. Triage de vulnerabilidad
125. Contención
126. Erradicación
127. Recuperación
128. Post-incident review
129. Cadena de custodia
130. Preservación de evidencia
131. Reloj y timeline
132. Comunicación interna
133. Comunicación a jugadores
134. Coordinación con Valve
135. Coordinación con proveedor
136. Brecha de datos
137. Credencial comprometida
138. Repositorio comprometido
139. Paquete malicioso
140. Build alterada
141. Malware o ransomware
142. Pérdida de dispositivo
143. Publicación accidental
144. Secretos en commit
145. Save exploit crítico

146. Denegación de servicio futura
147. Abuso de comunidad
148. Plan de crisis
149. Business continuity
150. RTO y RPO
151. Single-person dependency
152. Recuperación de cuentas
153. Contactos de emergencia
154. Simulacros
155. Métricas de seguridad
156. Security debt
157. Work packages
158. Gates
159. Checklist pre-commit
160. Checklist pre-build
161. Checklist pre-release
162. Plantilla de incidente
163. Plantilla de vulnerability report
164. Plantilla de risk acceptance
165. RACI
166. Definition of Ready
167. Definition of Done
168. Riesgos abiertos
169. Glosario
170. Fuentes oficiales
171. Inventario histórico
172. Historial de cambios

27 — Security and Incident Response Plan

Proyecto: Cartridge & Cloud

Estudio / Owner: VRM Games / Blas Luis Rocha González

Tecnología: Unity 6.3 LTS 6000.3.18f1, URP 17.3.0, C# 9 / netstandard2.1

Plataforma objetivo: Windows x64 / PC Steam

Versión de aplicación observada: 0.0.17

Fecha de consolidación: 2026-07-01

Estado: OPERATIONAL PLAN / IMPLEMENTATION PARTIAL / PUBLIC RELEASE BLOCKED

Este documento integra la genealogía histórica y el estado técnico observado. No es una certificación, un informe forense ni una garantía de ausencia de vulnerabilidades. Las capacidades futuras continúan NOT OPEN hasta superar sus gates.

Cómo leer los estados

- **IMPLEMENTED**: existe código o configuración.
- **VERIFIED**: existe prueba reproducible para la versión observada.
- **PARTIAL**: existe una parte, faltan controles o evidencia.
- **PLANNED**: política definida, ejecución pendiente.
- **NOT OPEN**: fuera de alcance y no autorizado.
- **BLOCKED**: no puede promoverse al gate indicado.

Resumen de postura

La inspección de 457 scripts C# no encontró llamadas de red, Steamworks, SDKs de analytics/crash reporting, `DllImport`, bloques `unsafe` ni patrones evidentes de secretos. Se localizaron 15 llamadas a `File.Delete` en 8 archivos y 7 llamadas a `Directory.Delete` en 7 archivos, principalmente en persistencia, recuperación y escenarios técnicos. El proyecto declara 41 dependencias directas y 57 entradas en `packages-lock.json`.

La seguridad actual se beneficia del alcance offline, la ausencia de código de red y una persistencia defensiva; sus principales brechas son operativas: identidad, backups, supply chain monitoring, repository controls, incident channel, candidate build hygiene y ejercicios de recuperación.

0. Propósito, alcance y autoridad

Propósito

Este documento establece el sistema de seguridad y respuesta a incidentes de **Cartridge & Cloud**. Protege el código, repositorio, documentación, arte fuente, paquetes, estaciones de trabajo, cuentas, secretos futuros, saves, builds, materiales de publicación y continuidad de VRM Games. Integra prevención, detección, respuesta, recuperación y aprendizaje.

Autoridad

La autoridad documental sigue la jerarquía vigente del proyecto. Este plan complementa —y no sustituye—

09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 23_Legal_Credits_and_Licenses_Register.xlsx, 24_Steam_Publishing_Plan.md, 25_Post_Launch_and_Live_Operations_Plan.md y 26_Privacy_Data_and_Telemetry_Plan.md.

Cuando una cuestión sea de privacidad, licencia o publicación, prevalece el documento especializado correspondiente y este plan aporta el procedimiento de seguridad.

Alcance

Incluye desarrollo local, Git/GitHub, Unity/UPM, Blender y herramientas DCC, backups, builds Windows, futura cuenta Steamworks, futura SteamPipe, soporte y vulnerabilidades reportadas. No afirma que el proyecto esté certificado contra NIST, OWASP, ISO 27001 u otro estándar. Los marcos se utilizan como vocabulario y checklist proporcional.

Objetivos de seguridad

1. Evitar pérdida irreversible del proyecto y de las credenciales.
2. Evitar publicación de una build manipulada, no autorizada o no reproducible.
3. Mantener la integridad de saves, economía e inventario frente a corrupción accidental.
4. Detectar secretos, dependencias vulnerables y cambios críticos antes del release.
5. Responder a incidentes con mínima improvisación y máxima preservación de evidencia.
6. Mantener sistemas online, Cloud y telemetría en **NOT OPEN** hasta superar sus gates.
7. Permitir que otra persona autorizada pueda recuperar el proyecto en una emergencia.

Estado del documento

OPERATIONAL PLAN / IMPLEMENTATION PARTIAL. La creación de este archivo no cierra automáticamente work packages, riesgos ni H6. El documento se revalida antes de onboarding Steamworks, antes de la primera candidata pública, después de un incidente severo y cuando cambie el modelo de distribución o datos.

1. Genealogía histórica de seguridad

La seguridad del proyecto no nació como documento único. Se distribuyó en reglas de build, QA, legal, Steam, persistencia, handoff y ADR. Esta consolidación conserva esa genealogía y evita atribuir a versiones antiguas capacidades que todavía no existían.

Período	Fuente	Aportación de seguridad
v0.3	Build & Versioning v0.3	Introdujo build metadata, instalación limpia, logs, SHA-256, archivado y compatibilidad de saves.
v0.3	QA Testing Plan v0.3	Incluyó backup, archivo truncado, versión antigua y build candidata con hashes y known issues.
v0.3	Legal Register v0.3	Prohibió introducir recursos en build pública sin origen, licencia, uso comercial, atribución y evidencia.
v0.4	Build & Versioning v0.4	Añadió evidencia de las primeras builds Windows, checksum y pre-release técnica.
v0.5	Package Manifest v0.3	Declaró que los paquetes documentales no incluían credenciales ni datos personales sensibles.
v0.5	Build & Versioning v0.5	Separó builds locales de ZIP/checksum de hitos y mantuvo Builds/ fuera de Git.
v0.6	Build & Versioning v0.6	Mantuvo 0.0.17, congeló tags/releases salvo gate y dejó StoreInitial pendiente de conexión.
Sprint 14	ADR-0056 a ADR-0060	Formalizaron save v2 compatible, escritura validada, checksum/generación, backup-first recovery y two-phase restore.
Sprint 15	ADR-0063 a ADR-0066	Añadieron autosave idempotente, preferencias globales y exclusividad de input UI.
Sprint 16	ADR-0069	Coordinó el checkpoint/autosave de Phase 1.

Regla de consolidación

- Una política histórica se conserva aunque hoy esté ampliada.
- Una build **PASS** histórica demuestra su gate, no la seguridad de la working copy actual.
- Los checksums históricos prueban integridad del artefacto archivado, no autenticidad criptográfica del productor.
- El save SHA-256 detecta corrupción, no evita manipulación por un atacante local.
- La ausencia histórica de Steamworks, backend o Cloud se mantiene como estado, no como deuda implícita que deba abrirse.

- Los cambios de v0.3 a v0.6 se tratan como evolución de preproducción a integración; no se elimina ninguna fuente previa.

El inventario completo de fuentes relevantes aparece en el capítulo 171 con baseline, ruta, tamaño y SHA-256.

2. Estado de seguridad actual

Resumen ejecutivo

La inspección de 457 scripts C# no encontró llamadas de red, Steamworks, SDKs de analytics/crash reporting, `DllImport`, bloques `unsafe` ni patrones evidentes de secretos. Se localizaron 15 llamadas a `File.Delete` en 8 archivos y 7 llamadas a `Directory.Delete` en 7 archivos, principalmente en persistencia, recuperación y escenarios técnicos. El proyecto declara 41 dependencias directas y 57 entradas en `packages-lock.json`.

Área	Evidencia actual	Estado
Runtime de red	Cero coincidencias de <code>UnityWebRequest</code> , <code>HttpClient</code> , sockets o Steamworks en 457 scripts	NOT PRESENT
Analytics/crash SDK	Cero coincidencias de proveedores externos; Unity Analytics y Cloud Diagnostics aparecen desactivados	NOT PRESENT / SETTINGS AUDIT
Secretos	Cero patrones evidentes en el árbol inspeccionado; <code>.env</code> , keystores y JKS están ignorados	PARTIAL / HISTORY NOT SCANNED
Interop nativo	Cero <code>DllImport</code> y cero bloques <code>unsafe</code>	NOT PRESENT
Operaciones destructivas	15 <code>File.Delete</code> y 7 <code>Directory.Delete</code>	REVIEW REQUIRED
Persistencia	Envelope SHA-256, generación, primario, <code>.bak</code> , <code>.tmp</code> , <code>.recovery</code> , schema 2	IMPLEMENTED / SECURITY LIMITS DOCUMENTED
Paquetes	41 directos; 57 entradas lock; sin scoped registry personalizado demostrado	VERSIONED / VULNERABILITY MONITORING MISSING
Build profile	Assets/_Project/Scenes/Bootstrap.unity, Assets/_Project/Scenes/MainMenu.unity, Assets/_Project/Scenes/Store.unity, Assets/_Project/Scenes/TestLab.unity	TESTLAB INCLUDED; STOREINITIAL ABSENT
Repositorio	<code>.gitignore</code> / <code>.gitattributes</code> presentes	REMOTE SETTINGS NOT VERIFIED
Steamworks	Sin App ID, SDK, SteamPipe o branch pública	NOT OPEN
Public release	0 builds autorizadas en el registro legal	BLOCKED

Interpretación

El perfil local-first reduce amenazas de servidor, cuentas de jugador y exposición de API, pero no elimina riesgos de supply chain, pérdida del único workstation, compromiso de GitHub/Steamworks, secretos futuros, manipulación de build, ransomware, publicación accidental o corrupción de saves. El mayor riesgo actual es operativo: dependencia de una sola persona, ausencia de restore drills, ausencia de security tooling remoto demostrado y gates futuros todavía abiertos.

Prioridades inmediatas

1. Crear `SECURITY.md` y un canal privado antes de cualquier beta pública.
2. Activar MFA y documentar recovery para GitHub, correo, Unity y futuro Steamworks.
3. Ejecutar secret scan del historial Git y configurar prevención en push cuando esté disponible.
4. Establecer backup 3-2-1 y probar restauración del repositorio, arte fuente y documentación.
5. Crear SBOM/package audit y proceso de advisories.
6. Retirar `TestLab` de una candidata y conectar `StoreInitial` antes del gate H6.
7. Ejecutar un tabletop de credencial comprometida y otro de build dañina.

3. Principios Secure by Design

1. **Responsabilidad del productor.** El jugador no debe cargar con configuraciones inseguras, procesos de recuperación imposibles o información engañosa.
2. **Seguro por defecto.** Servicios online, telemetría, Cloud, mods, Workshop y SDKs externos permanecen desactivados hasta aprobación explícita.
3. **Menor superficie.** Eliminar paquetes, herramientas, escenas y privilegios innecesarios es preferible a proteger complejidad que el producto no necesita.
4. **Mínimo privilegio.** Cada cuenta, colaborador, token y script recibe únicamente el permiso y la duración requeridos.
5. **Defensa en profundidad.** `.gitignore`, MFA, secret scanning, branch protection, backups y revocación cubren fallos distintos; ninguno sustituye a los demás.
6. **Fail-safe y atomicidad.** Ante error, no confirmar una transacción parcial, no reemplazar un save válido por uno inválido y no promover una build incierta.
7. **Trazabilidad.** Toda release y todo incidente deben poder relacionarse con commit, dependencias, build, hash, manifest, owner y decisión.
8. **Recuperación probada.** Backup sin restore test no se considera control cerrado.

9. **Transparencia.** Los estados `NOT OPEN`, `BLOCKED`, `PARTIAL` y `PASS` se comunican sin convertir intención en realidad.

10. **Aprendizaje.** Incidentes, near misses y findings actualizan controles, tests y documentación.

Estos principios se alinean de forma proporcional con NIST CSF 2.0, NIST SSDF y la orientación Secure by Design; no constituyen una certificación.

4. Gobernanza y responsabilidad

Owner y modelo de operación

El owner de seguridad es **Blas Luis Rocha González / VRM Games**. En una producción individual, el mismo owner puede programar, revisar y publicar, pero debe introducir controles compensatorios: ramas, pausa deliberada, diff limpio, test automatizado, build reproducible, checklist firmada y conservación de `previous_stable`.

Responsabilidades mínimas

- Mantener inventario de activos, cuentas, paquetes y secretos futuros.
- Revisar advisories y vulnerabilidades relevantes.
- Autorizar excepciones y aceptar riesgo residual por escrito.
- Declarar incident commander durante SEC-0/SEC-1.
- Separar evidencia técnica de comunicación pública.
- Solicitar apoyo externo legal, forense o técnico cuando la capacidad interna sea insuficiente.

Separación de funciones proporcional

Para una release pública, se recomienda que al menos una segunda persona de confianza revise el Go/No-Go, permisos Steamworks, store/build selection y plan de rollback. Si no existe segundo revisor, la release se retrasa o se usa una revisión externa puntual; no se considera equivalente una relectura inmediata del propio autor.

Revisión

El plan se revisa trimestralmente durante desarrollo activo, antes de onboarding Steamworks, tras cambios de cuenta/proveedor, antes de una candidata pública y después de todo incidente SEC-0 a SEC-2.

5. Modelo de amenazas

Actores considerados

- Malware oportunista, ransomware y phishing.
- Atacante que obtiene una cuenta de correo, GitHub, Unity o Steamworks.
- Dependencia, plugin, extensión o archivo DCC comprometido.
- Colaborador o proveedor con exceso de permisos.
- Jugador que manipula saves locales o entradas para provocar estados inválidos.
- Error humano: publicación accidental, borrado, merge incorrecto o filtración de secreto.
- Fallo de proveedor, almacenamiento, hardware o sincronización.

Objetivos del adversario o del fallo

Robar código/arte, insertar malware, distribuir una build alterada, secuestrar una cuenta, destruir backups, extraer datos de soporte, manipular economía, provocar corrupción, suplantar al estudio o extorsionar.

Supuestos

El runtime actual no posee backend ni datos de cuenta, por lo que el foco se coloca en desarrollo, supply chain, distribución y saves. Este modelo cambia de forma sustancial al abrir Steam Cloud, telemetría, cuentas, Workshop, web propia o servicios de soporte.

Método

Cada nueva feature identifica activos, entradas, trust boundaries, abuso, impacto, controles, detección, recuperación y owner. El resultado se enlaza al risk register; una amenaza no mitigada puede cerrar el gate aunque la feature funcione técnicamente.

6. Activos que proteger

Clase	Ejemplos	Impacto principal
Propiedad intelectual	Código, GDD, arte, audio, modelos, branding	Pérdida, filtración o disputa de autoría

Clase	Ejemplos	Impacto principal
Fuente reproducible	Assets, Packages, ProjectSettings, Tools, documentación	Imposibilidad de reconstruir una build
Identidad	Email, GitHub, Unity, Steamworks, dominio futuro	Suplantación y publicación no autorizada
Secretos	Tokens, passwords, recovery codes, VDF con credenciales	Acceso persistente a cuentas o pipeline
Builds	ZIP, ejecutable, depots, manifests, previous stable	Distribución de malware o versión dañina
Datos locales	Saves, backups, preferencias, logs	Pérdida de progreso o exposición en soporte
Evidencia	Logs, hashes, screenshots, timeline, incident records	Incapacidad de investigar o demostrar acciones
Continuidad	Backups, contactos, instrucciones de restore	Parada del proyecto por fallo único

Cada activo recibe ID, owner, ubicación primaria, clasificación, dependencias, backup, RPO/RTO y gate de distribución. Los conceptos y assets **VISION / NOT OPEN** también se protegen para evitar publicación accidental, aunque no formen parte de la candidata.

7. Superficies de ataque

Las superficies actuales son el workstation Windows, navegador/correo, Git/GitHub, Unity Hub/Editor/UPM, Visual Studio, Blender, archivos descargados, almacenamiento de backups y canales de documentación. Las superficies futuras incluyen Steamworks, SteamPipe, soporte público, website, Cloud, analytics y crash reporting.

El runtime offline recibe inputs de usuario, archivos JSON locales, PlayerPrefs y assets empaquetados. Aunque no exponga sockets, debe soportar archivos corruptos, paths inesperados, interrupciones durante escritura y valores fuera de rango. El Editor posee más privilegios que el Player: editor scripts, importers y plugins se tratan como código ejecutable.

Toda apertura de superficie requiere owner, threat model, provider review, least privilege, logging mínimo, shutdown plan y test de recuperación.

8. Límites de confianza

1. **Internet → workstation:** descargas, email, navegador, package registry y archivos externos no son confiables por defecto.
2. **Proveedor → repositorio:** un paquete oficial sigue siendo tercero y puede cambiar, quedar vulnerable o comprometerse.
3. **Editor → Player:** código/editor tooling no debe filtrarse a la build pública.

4. **Repositorio → build:** una build solo es confiable si procede de commit, dependencias y script identificados.
5. **Build → Steam:** upload y promoción requieren credencial, branch, manifest y decisión separadas.
6. **Jugador → save/log:** archivos locales pueden estar corruptos o manipulados; soporte no debe tratarlos como datos confiables ni seguros.
7. **Canal público → incidente:** un reporte puede ser genuino, incompleto, malicioso o contener datos personales; se preserva y valida.

Los límites se documentan en arquitectura y se revisan al añadir una integración o cambiar el flujo de datos.

9. Clasificación de información

Nivel	Ejemplos	Tratamiento
PUBLIC	Store copy aprobado, patch notes, press kit	Publicable tras revisión
INTERNAL	Roadmap, QA, documentación de trabajo	Compartición limitada al proyecto
CONFIDENTIAL	Build no anunciada, contratos, incidentes abiertos	Acceso por necesidad
SECRET	Passwords, tokens, recovery codes, claves de firma	Password manager/HSM; nunca repositorio
PERSONAL/RESTRICTED	Datos fiscales, tickets, saves/logs de jugadores	Plan de Privacidad, minimización y retención

La clasificación acompaña al activo, no al formato. Una captura puede contener un token; un log puede contener una ruta de usuario; un VDF puede ser una plantilla inocua o incluir credenciales. Antes de compartir se revisa contenido efectivo, metadatos y audiencia.

10. Código fuente

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Código fuente**, la decisión por defecto es **PARTIAL / DOCUMENTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El inventario contiene 457 scripts C# y 12 asmdefs; la suite aceptada es `1215 EditMode + 70 PlayMode`. No se ha demostrado revisión de seguridad independiente.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Revisar cambios de persistencia, paths, economía, serialización, editor scripts y composición runtime con checklist específica.
6. Para trabajo individual, compensar la falta de segundo revisor con branch, diff limpio, tests, pausa deliberada y revisión posterior.

Evidencia y criterio de gate

- Diff, commit, test run y checklist de revisión.
- ADR cuando cambia una invariante o trust boundary.

Criterio PASS de Código fuente: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Normalizar revisión superficial por ser un proyecto individual.
- Introducir una regresión de integridad en un cambio aparentemente local.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

11. Repositorios y ramas

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Repositorios y ramas**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El paquete aporta `.gitignore` y `.gitattributes`, pero no demuestra reglas de protección de rama, Actions, Dependabot, secret scanning o `SECURITY.md` configurados en el repositorio remoto.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Proteger `main`, exigir checks y evitar force-push en ramas de release cuando la plataforma lo permita.
6. Mantener una rama corta por cambio y promover únicamente tras validación.

Evidencia y criterio de gate

- Captura/export de settings de repositorio.
- Historial de PR o registro de revisión compensatoria.

Criterio PASS de Repositorios y ramas: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Pérdida de historial por force-push.
- Promoción directa de código no probado o de una cuenta comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

12. Credenciales y secretos

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Credenciales y secretos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Credenciales y secretos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

13. Steamworks y cuentas de partner

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Steamworks y cuentas de partner**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Steamworks y cuentas de partner: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`.

14. Unity ID y servicios

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Unity ID y servicios**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Unity ID y servicios: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.

- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

15. GitHub y dispositivos

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **GitHub y dispositivos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El paquete aporta `.gitignore` y `.gitattributes`, pero no demuestra reglas de protección de rama, Actions, Dependabot, secret scanning o `SECURITY.md` configurados en el repositorio remoto.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Proteger `main`, exigir checks y evitar force-push en ramas de release cuando la plataforma lo permita.
6. Mantener una rama corta por cambio y promover únicamente tras validación.

Evidencia y criterio de gate

- Captura/export de settings de repositorio.
- Historial de PR o registro de revisión compensatoria.

Criterio PASS de GitHub y dispositivos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Pérdida de historial por force-push.
- Promoción directa de código no probado o de una cuenta comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

16. Estaciones de trabajo

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Estaciones de trabajo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Estaciones de trabajo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

17. Blender y herramientas DCC

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Blender y herramientas DCC**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen herramientas y conceptos generados, además de fuentes `.blend` y modelos; no se ha demostrado sandbox o revisión automática de scripts/macros embebidos.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Abrir archivos externos en una copia y revisar scripts, drivers, plugins y macros antes de confiar en ellos.
6. Conservar fuente, hash, procedencia y versión de herramienta.
7. No ejecutar archivos descargados con privilegios elevados.

Evidencia y criterio de gate

- Registro de procedencia y hash del archivo original.
- Resultado de escaneo y revisión de scripts/plugins.

Criterio PASS de Blender y herramientas DCC: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ejecución de código desde un asset o plugin.

- Contaminar el proyecto con una fuente de procedencia ambigua.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

18. Paquetes y dependencias

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Paquetes y dependencias**, la decisión por defecto es **PARTIAL / DOCUMENTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.

7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Paquetes y dependencias: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `23_Legal_Credits_and_Licenses_Register.xlsx`.

19. Supply chain de Unity Package Manager

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Supply chain de Unity Package Manager**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Supply chain de Unity Package Manager: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

20. SBOM

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **SBOM**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.

- Test run y build de compatibilidad tras la actualización.

Criterio PASS de SBOM: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

21. Licencias y seguridad

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Licencias y seguridad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El registro legal mantiene revisión de LICENSE/NOTICE pendiente por paquete y bloquea distribución pública mientras no se cierre.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.

3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Tratar licencia, procedencia y vulnerabilidad como dimensiones separadas.
6. Conservar texto legal exacto por versión y actualizar notices al cambiar paquetes.

Evidencia y criterio de gate

- Registro 23 y archivo de notices generado para la candidata.
- Hash y versión de cada dependencia distribuida.

Criterio PASS de Licencias y seguridad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Distribuir software sin cumplir avisos.
- Confundir paquete oficial con licencia automáticamente resuelta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `23_Legal_Credits_and_Licenses_Register.xlsx`.

22. Artefactos de build

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Artefactos de build**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Artefactos de build: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`.

23. Integridad y checksums

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Integridad y checksums**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Integridad y checksums: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

24. Firmas y procedencia

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Firmas y procedencia**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.

7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Firmas y procedencia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

25. Backups del repositorio

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Backups del repositorio**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Backups del repositorio: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

26. Backups documentales

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Backups documentales**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Backups documentales: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

27. Backups de herramientas y fuentes

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Backups de herramientas y fuentes**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.

6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Backups de herramientas y fuentes: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

28. Recuperación ante pérdida total

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Recuperación ante pérdida total**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario. Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.
9. Preservar copia antes de modificar el sistema afectado.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.
- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Recuperación ante pérdida total: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.

- Imposibilidad de continuar por dependencia de una sola persona.
- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.
- Convertir la documentación en cumplimiento de papel.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

29. Control de acceso

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Control de acceso**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.

7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.

Criterio PASS de Control de acceso: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

30. Principio de mínimo privilegio

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Principio de mínimo privilegio**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.

Criterio PASS de Principio de mínimo privilegio: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

31. MFA

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **MFA**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de MFA: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.

- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

32. Altas, bajas y cambios de acceso

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Altas, bajas y cambios de acceso**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.

Criterio PASS de Altas, bajas y cambios de acceso: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

33. Roles de colaborador

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Roles de colaborador**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.

Criterio PASS de Roles de colaborador: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

34. Acceso de proveedores

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Acceso de proveedores**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa. No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.
8. Separar facts, hipótesis, impacto, acciones y próxima actualización.
9. Coordinar mensajes técnicos, legales, privacidad y comunidad.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.
- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Acceso de proveedores: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.
- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

35. Cuentas compartidas

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Cuentas compartidas**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.

2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Cuentas compartidas: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

36. Password manager

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Password manager**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Password manager: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.

- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

37. Rotación de secretos

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Rotación de secretos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Rotación de secretos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

38. Secret scanning

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Secret scanning**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera **PLANNED** hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Secret scanning: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

39. Variables de entorno

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Variables de entorno**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Variables de entorno: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.

- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

40. Archivos VDF y SteamPipe

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Archivos VDF y SteamPipe**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados. `EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.

7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.
8. Generar build desde script reproducible y lista de escenas explícita.
9. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.
- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Archivos VDF y SteamPipe: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.
- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`.

41. API keys futuras

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **API keys futuras**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de API keys futuras: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.

- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

42. Configuración segura por defecto

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Configuración segura por defecto**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Package Manager tiene preview packages desactivados y registry oficial; Version Control usa Visible Meta Files. `UnityConnectSettings` desactiva Analytics y Cloud Diagnostics, pero mantiene `EngineDiagnosticsEnabled: 1`; `ProjectSettings.asset` contiene `submitAnalytics: 1`, por lo que debe auditarse el comportamiento real sin asumir transmisión. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Revisar diffs de `ProjectSettings` y `Packages` como cambios de infraestructura.

6. Documentar settings ambiguos y validar con build/red observada cuando sean relevantes.
7. No activar servicios cloud o diagnostics por accidente al abrir el Editor.
8. Usar IDs estables, owner, target, evidencia y estado.
9. Cerrar deuda solo con prueba; una aceptación necesita vencimiento y riesgo residual.

Evidencia y criterio de gate

- Snapshot de settings y prueba de red/servicio si se abre el gate.
- Diff aprobado y resultado de build.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Configuración segura por defecto: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Activar recopilación o servicio no documentado.
- Cambiar escenas, identificador o build flags sin revisión.
- Convertir la documentación en cumplimiento de papel.
- Mantener controles obsoletos sin ejecutar.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

43. Hardening de Windows de desarrollo

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Hardening de Windows de desarrollo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Hardening de Windows de desarrollo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

44. Actualizaciones del sistema

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Actualizaciones del sistema**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas. `manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.

4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.
9. Versionar manifest y lockfile; revisar cambios directos y transitivos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.
- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Actualizaciones del sistema: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.
- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: [09_CSharp_Coding_Standards.md](#), [11_Build_y_Versioning_Guide.md](#), [13_Trazabilidad_y_Control_de_Cambios.xlsx](#).

45. Antimalware y falsos positivos

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Antimalware y falsos positivos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas. El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.
9. Detener propagación y acceso sin borrar evidencia.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.
- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Antimalware y falsos positivos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.
- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

46. Cifrado de dispositivo

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Cifrado de dispositivo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.

2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Cifrado de dispositivo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: [09_CSharp_Coding_Standards.md](#), [11_Build_y_Versioning_Guide.md](#), [13_Trazabilidad_y_Control_de_Cambios.xlsx](#).

47. Bloqueo y sesión

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Bloqueo y sesión**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Bloqueo y sesión: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

48. Medios extraíbles

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Medios extraíbles**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.

6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Medios extraíbles: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

49. Trabajo remoto

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Trabajo remoto**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Trabajo remoto: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

50. Wi-Fi y red

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Wi-Fi y red**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Wi-Fi y red: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

51. Descargas y fuentes confiables

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Descargas y fuentes confiables**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen herramientas y conceptos generados, además de fuentes `.blend` y modelos; no se ha demostrado sandbox o revisión automática de scripts/macros embebidos.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Abrir archivos externos en una copia y revisar scripts, drivers, plugins y macros antes de confiar en ellos.

6. Conservar fuente, hash, procedencia y versión de herramienta.
7. No ejecutar archivos descargados con privilegios elevados.

Evidencia y criterio de gate

- Registro de procedencia y hash del archivo original.
- Resultado de escaneo y revisión de scripts/plugins.

Criterio PASS de Descargas y fuentes confiables: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ejecución de código desde un asset o plugin.
- Contaminar el proyecto con una fuente de procedencia ambigua.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

52. Macros y documentos

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Macros y documentos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen herramientas y conceptos generados, además de fuentes `.blend` y modelos; no se ha demostrado sandbox o revisión automática de scripts/macros embebidos.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Abrir archivos externos en una copia y revisar scripts, drivers, plugins y macros antes de confiar en ellos.
6. Conservar fuente, hash, procedencia y versión de herramienta.
7. No ejecutar archivos descargados con privilegios elevados.

Evidencia y criterio de gate

- Registro de procedencia y hash del archivo original.
- Resultado de escaneo y revisión de scripts/plugins.

Criterio PASS de Macros y documentos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ejecución de código desde un asset o plugin.
- Contaminar el proyecto con una fuente de procedencia ambigua.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

53. Contenido generado y archivos externos

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Contenido generado y archivos externos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen herramientas y conceptos generados, además de fuentes `.blend` y modelos; no se ha demostrado sandbox o revisión automática de scripts/macros embebidos.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Abrir archivos externos en una copia y revisar scripts, drivers, plugins y macros antes de confiar en ellos.
6. Conservar fuente, hash, procedencia y versión de herramienta.
7. No ejecutar archivos descargados con privilegios elevados.

Evidencia y criterio de gate

- Registro de procedencia y hash del archivo original.
- Resultado de escaneo y revisión de scripts/plugins.

Criterio PASS de Contenido generado y archivos externos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ejecución de código desde un asset o plugin.

- Contaminar el proyecto con una fuente de procedencia ambigua.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

54. Importación de Unity packages

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Importación de Unity packages**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.

7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Importación de Unity packages: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

55. Editor scripts y ejecución automática

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Editor scripts y ejecución automática**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Editor scripts y ejecución automática: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

56. Build scripts

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Build scripts**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.

- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Build scripts: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`.

57. CI/CD futuro

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **CI/CD futuro**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de CI/CD futuro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

58. Protección de main

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Protección de main**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El paquete aporta `.gitignore` y `.gitattributes`, pero no demuestra reglas de protección de rama, Actions, Dependabot, secret scanning o `SECURITY.md` configurados en el repositorio remoto.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Proteger `main`, exigir checks y evitar force-push en ramas de release cuando la plataforma lo permita.
6. Mantener una rama corta por cambio y promover únicamente tras validación.

Evidencia y criterio de gate

- Captura/export de settings de repositorio.
- Historial de PR o registro de revisión compensatoria.

Criterio PASS de Protección de main: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Pérdida de historial por force-push.
- Promoción directa de código no probado o de una cuenta comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

59. Code review

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Code review**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El inventario contiene 457 scripts C# y 12 asmdefs; la suite aceptada es `1215 EditMode + 70 PlayMode`. No se ha demostrado revisión de seguridad independiente.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Revisar cambios de persistencia, paths, economía, serialización, editor scripts y composición runtime con checklist específica.
6. Para trabajo individual, compensar la falta de segundo revisor con branch, diff limpio, tests, pausa deliberada y revisión posterior.

Evidencia y criterio de gate

- Diff, commit, test run y checklist de revisión.
- ADR cuando cambia una invariante o trust boundary.

Criterio PASS de Code review: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Normalizar revisión superficial por ser un proyecto individual.
- Introducir una regresión de integridad en un cambio aparentemente local.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

60. Change review de ProjectSettings

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Change review de ProjectSettings**, la decisión por defecto es **PARTIAL / DOCUMENTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Package Manager tiene preview packages desactivados y registry oficial; Version Control usa Visible Meta Files.

`UnityConnectSettings` desactiva Analytics y Cloud Diagnostics, pero mantiene `EngineDiagnosticsEnabled: 1`;

`ProjectSettings.asset` contiene `submitAnalytics: 1`, por lo que debe auditarse el comportamiento real sin asumir

transmisión.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Revisar diffs de ProjectSettings y Packages como cambios de infraestructura.
6. Documentar settings ambiguos y validar con build/red observada cuando sean relevantes.
7. No activar servicios cloud o diagnostics por accidente al abrir el Editor.

Evidencia y criterio de gate

- Snapshot de settings y prueba de red/servicio si se abre el gate.
- Diff aprobado y resultado de build.

Criterio PASS de Change review de ProjectSettings: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Activar recopilación o servicio no documentado.
- Cambiar escenas, identificador o build flags sin revisión.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

61. Change review de Packages

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Change review de Packages**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Package Manager tiene preview packages desactivados y registry oficial; Version Control usa Visible Meta Files.

`UnityConnectSettings` desactiva Analytics y Cloud Diagnostics, pero mantiene `EngineDiagnosticsEnabled: 1`;

`ProjectSettings.asset` contiene `submitAnalytics: 1`, por lo que debe auditarse el comportamiento real sin asumir transmisión.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Revisar diffs de ProjectSettings y Packages como cambios de infraestructura.
6. Documentar settings ambiguos y validar con build/red observada cuando sean relevantes.
7. No activar servicios cloud o diagnostics por accidente al abrir el Editor.

Evidencia y criterio de gate

- Snapshot de settings y prueba de red/servicio si se abre el gate.
- Diff aprobado y resultado de build.

Criterio PASS de Change review de Packages: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Activar recopilación o servicio no documentado.
- Cambiar escenas, identificador o build flags sin revisión.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

62. Dependency pinning

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Dependency pinning**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.

5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Dependency pinning: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `23_Legal_Credits_and_Licenses_Register.xlsx`.

63. Evaluación de actualizaciones

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Evaluación de actualizaciones**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Evaluación de actualizaciones: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

64. Vulnerabilidades conocidas

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Vulnerabilidades conocidas**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera **PLANNED** hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Vulnerabilidades conocidas: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

65. CVE y advisories

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **CVE y advisories**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.

5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de CVE y advisories: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `23_Legal_Credits_and_Licenses_Register.xlsx`.

66. Política de parcheo

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Política de parcheo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Política de parcheo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

67. Ventanas de mantenimiento

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Ventanas de mantenimiento**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera **PLANNED** hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Ventanas de mantenimiento: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

68. Excepciones de seguridad

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Excepciones de seguridad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.

6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Excepciones de seguridad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

69. Aceptación de riesgo

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Aceptación de riesgo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera **PLANNED** hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Aceptación de riesgo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

70. Secure coding C#

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Secure coding C#**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El inventario contiene 457 scripts C# y 12 asmdefs; la suite aceptada es `1215 EditMode + 70 PlayMode`. No se ha demostrado revisión de seguridad independiente.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Revisar cambios de persistencia, paths, economía, serialización, editor scripts y composición runtime con checklist específica.
6. Para trabajo individual, compensar la falta de segundo revisor con branch, diff limpio, tests, pausa deliberada y revisión posterior.

Evidencia y criterio de gate

- Diff, commit, test run y checklist de revisión.
- ADR cuando cambia una invariante o trust boundary.

Criterio PASS de Secure coding C#: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Normalizar revisión superficial por ser un proyecto individual.
- Introducir una regresión de integridad en un cambio aparentemente local.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

71. Validación de entrada

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Validación de entrada**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Validación de entrada: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

72. Paths y traversal

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Paths y traversal**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Paths y traversal: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

73. Deserialización y JSON

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Deserialización y JSON**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Deserialización y JSON: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

74. Archivos temporales

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Archivos temporales**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.

7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Archivos temporales: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

75. Atomicidad de saves

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Atomicidad de saves**, la decisión por defecto es **PARTIAL / IMPLEMENTED AND TESTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Atomicidad de saves: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

76. Corrupción y recuperación

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Corrupción y recuperación**, la decisión por defecto es **PARTIAL / IMPLEMENTED AND TESTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local. Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. Preservar copia antes de modificar el sistema afectado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Timeline UTC, hashes, capturas, logs y lista de acciones.

- Postmortem sin culpabilización y acciones con owner.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Corrupción y recuperación: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.
- Convertir la documentación en cumplimiento de papel.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

77. Integer overflow y economía

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Integer overflow y economía**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La economía usa unidades menores e invariantes atómicas; los ADR históricos protegen ledger, reservas, receiving, checkout y cierres diarios.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Validar rangos antes de sumar, multiplicar o convertir.
6. Usar identificadores idempotentes para operaciones repetibles.
7. No confiar en valores cargados hasta validarlos contra invariantes.

Evidencia y criterio de gate

- Tests de límites, repetición, rollback y ledger.
- Snapshot inválido rechazado sin mutación parcial.

Criterio PASS de Integer overflow y economía: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Desbordamiento, saldo negativo o duplicación.
- Repetición de una operación tras crash o retry.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

78. Idempotencia

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Idempotencia**, la decisión por defecto es **PARTIAL / IMPLEMENTED AND TESTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La economía usa unidades menores e invariantes atómicas; los ADR históricos protegen ledger, reservas, receiving, checkout y cierres diarios.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Validar rangos antes de sumar, multiplicar o convertir.
6. Usar identificadores idempotentes para operaciones repetibles.
7. No confiar en valores cargados hasta validarlos contra invariantes.

Evidencia y criterio de gate

- Tests de límites, repetición, rollback y ledger.
- Snapshot inválido rechazado sin mutación parcial.

Criterio PASS de Idempotencia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Desbordamiento, saldo negativo o duplicación.

- Repetición de una operación tras crash o retry.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

79. Excepciones y fail-safe

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Excepciones y fail-safe**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Excepciones y fail-safe: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

80. Logging seguro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Logging seguro**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Logging seguro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `26_Privacy_Data_and_Telemetry_Plan.md`.

81. Redacción de logs

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Redacción de logs**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Redacción de logs: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.

- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `26_Privacy_Data_and_Telemetry_Plan.md`.

82. Dumps y datos sensibles

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Dumps y datos sensibles**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Dumps y datos sensibles: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `26_Privacy_Data_and_Telemetry_Plan.md`.

83. Network code futuro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Network code futuro**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece **NOT OPEN**.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Network code futuro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

84. TLS y certificados

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **TLS y certificados**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece **NOT OPEN**.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de TLS y certificados: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.

- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

85. Backend futuro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Backend futuro**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece **NOT OPEN**.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Backend futuro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

86. Autenticación futura

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Autenticación futura**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece **NOT OPEN**.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Autenticación futura: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

87. Autorización futura

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Autorización futura**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece **NOT OPEN**.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Autorización futura: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.

- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

88. Steam ID futuro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Steam ID futuro**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece **NOT OPEN**.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Steam ID futuro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`, `26_Privacy_Data_and_Telemetry_Plan.md`.

89. Anti-cheat y fraude

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Anti-cheat y fraude**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El juego es single-player offline; no existe economía real, competición, leaderboard ni Workshop. El save local puede ser modificado por el propietario del equipo.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No prometer inviolabilidad del save local.
6. Proteger invariantes y recuperación, no desplegar DRM/anti-cheat desproporcionado.
7. Reevaluar al introducir logros, rankings, Cloud, Workshop o contenido compartido.

Evidencia y criterio de gate

- Decisión de alcance y tests de save inválido.
- Threat model actualizado antes de abrir sistemas competitivos.

Criterio PASS de Anti-cheat y fraude: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Penalizar al jugador offline sin beneficio real.
- Reutilizar datos locales manipulables en un futuro sistema competitivo.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

90. Manipulación de saves

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Manipulación de saves**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local. El juego es single-player offline; no existe economía real, competición, leaderboard ni Workshop. El save local puede ser modificado por el propietario del equipo.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. No prometer inviolabilidad del save local.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Decisión de alcance y tests de save inválido.
- Threat model actualizado antes de abrir sistemas competitivos.

Criterio PASS de Manipulación de saves: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Penalizar al jugador offline sin beneficio real.
- Reutilizar datos locales manipulables en un futuro sistema competitivo.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

91. Integridad de economía offline

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Integridad de economía offline**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`. La economía usa unidades menores e invariantes atómicas; los ADR históricos protegen ledger, reservas, receiving, checkout y cierres diarios. El juego es single-player offline; no existe economía real, competición, leaderboard ni Workshop. El save local puede ser modificado por el propietario del equipo.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.
9. Validar rangos antes de sumar, multiplicar o convertir.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.
- Tests de límites, repetición, rollback y ledger.
- Snapshot inválido rechazado sin mutación parcial.
- Decisión de alcance y tests de save inválido.
- Threat model actualizado antes de abrir sistemas competitivos.

Criterio PASS de Integridad de economía offline: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.
- Desbordamiento, saldo negativo o duplicación.
- Repetición de una operación tras crash o retry.
- Penalizar al jugador offline sin beneficio real.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

92. Modding y Workshop futuros

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Modding y Workshop futuros**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El juego es single-player offline; no existe economía real, competición, leaderboard ni Workshop. El save local puede ser modificado por el propietario del equipo.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No prometer inviolabilidad del save local.
6. Proteger invariantes y recuperación, no desplegar DRM/anti-cheat desproporcionado.
7. Reevaluar al introducir logros, rankings, Cloud, Workshop o contenido compartido.

Evidencia y criterio de gate

- Decisión de alcance y tests de save inválido.
- Threat model actualizado antes de abrir sistemas competitivos.

Criterio PASS de Modding y Workshop futuros: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Penalizar al jugador offline sin beneficio real.
- Reutilizar datos locales manipulables en un futuro sistema competitivo.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

93. Plugins nativos

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Plugins nativos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.

3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Plugins nativos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

94. Native DLLs

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Native DLLs**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Native DLLs: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

95. Signing de ejecutables

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Signing de ejecutables**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.

7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Signing de ejecutables: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

96. Reputación y SmartScreen

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Reputación y SmartScreen**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Reputación y SmartScreen: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

97. Installer y distribución

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Installer y distribución**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Installer y distribución: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`.

98. Steam depots y branches

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Steam depots y branches**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El paquete aporta `.gitignore` y `.gitattributes`, pero no demuestra reglas de protección de rama, Actions, Dependabot, secret scanning o `SECURITY.md` configurados en el repositorio remoto. `EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.

5. Proteger `main`, exigir checks y evitar force-push en ramas de release cuando la plataforma lo permita.
6. Mantener una rama corta por cambio y promover únicamente tras validación.
7. Generar build desde script reproducible y lista de escenas explícita.
8. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
9. Promover una misma candidata validada; no recompilar después del Go.

Evidencia y criterio de gate

- Captura/export de settings de repositorio.
- Historial de PR o registro de revisión compensatoria.
- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Steam depots y branches: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Pérdida de historial por force-push.
- Promoción directa de código no probado o de una cuenta comprometida.
- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`.

99. Rollback seguro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Rollback seguro**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Rollback seguro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `25_Post_Launch_and_Live_Operations_Plan.md`.

100. Previous stable

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Previous stable**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Generar build desde script reproducible y lista de escenas explícita.

6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Previous stable: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `25_Post_Launch_and_Live_Operations_Plan.md`.

101. Release candidate

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Release candidate**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Release candidate: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`.

102. Reproducibilidad

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Reproducibilidad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Reproducibilidad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

103. Hash manifest

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Hash manifest**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión `0.0.17`.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.

4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Hash manifest: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

104. THIRD_PARTY_NOTICES

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **THIRD_PARTY_NOTICES**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El registro legal mantiene revisión de LICENSE/NOTICE pendiente por paquete y bloquea distribución pública mientras no se cierre.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Tratar licencia, procedencia y vulnerabilidad como dimensiones separadas.
6. Conservar texto legal exacto por versión y actualizar notices al cambiar paquetes.

Evidencia y criterio de gate

- Registro 23 y archivo de notices generado para la candidata.
- Hash y versión de cada dependencia distribuida.

Criterio PASS de THIRD_PARTY_NOTICES: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Distribuir software sin cumplir avisos.
- Confundir paquete oficial con licencia automáticamente resuelta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `23_Legal_Credits_and_Licenses_Register.xlsx`.

105. Security test plan

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Security test plan**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Security test plan: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

106. Static analysis

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Static analysis**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.

7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Static analysis: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

107. Dependency scanning

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Dependency scanning**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada. La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.
9. Añadir pruebas por riesgo y no por herramienta.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.
- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Dependency scanning: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.
- Falsa sensación de seguridad por cero findings.

- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `23_Legal_Credits_and_Licenses_Register.xlsx`.

108. Secret scanning automatizado

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Secret scanning automatizado**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Secret scanning automatizado: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

109. Malware scanning de artefactos

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Malware scanning de artefactos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada. El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.
9. Detener propagación y acceso sin borrar evidencia.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.
- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Malware scanning de artefactos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.
- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

110. Fuzzing selectivo

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Fuzzing selectivo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Fuzzing selectivo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

111. Tests de paths y archivos

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests de paths y archivos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local. La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.

3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. Añadir pruebas por riesgo y no por herramienta.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Tests de paths y archivos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

112. Tests de corrupción

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests de corrupción**, la decisión por defecto es **PARTIAL / IMPLEMENTED AND TESTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local. La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. Añadir pruebas por riesgo y no por herramienta.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Tests de corrupción: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

113. Tests de rollback

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests de rollback**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada. La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.
9. Añadir pruebas por riesgo y no por herramienta.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.
- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Tests de rollback: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.
- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 25_Post_Launch_and_Live_Operations_Plan.md.

114. Tests de permisos

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests de permisos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Tests de permisos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

115. Tests en usuario estándar

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests en usuario estándar**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera **PLANNED** hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Tests en usuario estándar: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

116. Clean machine test

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Clean machine test**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Clean machine test: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

117. Proton/Deck security review

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Proton/Deck security review**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La baseline funcional es `1215 EditMode + 70 PlayMode`; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Proton/Deck security review: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

118. Incident taxonomy

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Incident taxonomy**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, `SECURITY.md`, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.

7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Incident taxonomy: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

119. Severidades SEC-0 a SEC-4

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Severidades SEC-0 a SEC-4**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, `SECURITY.md`, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Severidades SEC-0 a SEC-4: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

120. Detección

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Detección**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, `SECURITY.md`, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Detección: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

121. Canales de reporte

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Canales de reporte**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, `SECURITY.md`, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.

7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Canales de reporte: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

122. Vulnerability disclosure

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Vulnerability disclosure**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, `SECURITY.md`, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Vulnerability disclosure: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

123. Recepción responsable de reportes

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Recepción responsable de reportes**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, `SECURITY.md`, bug bounty ni sistema de tickets. La taxonomía funcional S0–S4 no sustituye la severidad de seguridad SEC-0–SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Recepción responsable de reportes: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

124. Triage de vulnerabilidad

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Triage de vulnerabilidad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, `SECURITY.md`, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.

7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Triage de vulnerabilidad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

125. Contención

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Contención**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Contención: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

126. Erradicación

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Erradicación**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Erradicación: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

127. Recuperación

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Recuperación**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.

6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.
9. Usar IDs estables, owner, target, evidencia y estado.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Recuperación: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.
- Convertir la documentación en cumplimiento de papel.
- Mantener controles obsoletos sin ejecutar.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

128. Post-incident review

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Post-incident review**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Post-incident review: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

129. Cadena de custodia

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Cadena de custodia**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.

7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Cadena de custodia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

130. Preservación de evidencia

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Preservación de evidencia**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Preservación de evidencia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

131. Reloj y timeline

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Reloj y timeline**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Reloj y timeline: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

132. Comunicación interna

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Comunicación interna**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.

7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Comunicación interna: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `25_Post_Launch_and_Live_Operations_Plan.md`.

133. Comunicación a jugadores

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Comunicación a jugadores**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.
7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Comunicación a jugadores: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: [09_CSharp_Coding_Standards.md](#), [11_Build_y_Versioning_Guide.md](#), [13_Trazabilidad_y_Control_de_Cambios.xlsx](#), [25_Post_Launch_and_Live_Operations_Plan.md](#).

134. Coordinación con Valve

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Coordinación con Valve**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.
7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Coordinación con Valve: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

135. Coordinación con proveedor

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Coordinación con proveedor**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.

7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.

8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Coordinación con proveedor: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

136. Brecha de datos

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Brecha de datos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.
7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Brecha de datos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: [09_CSharp_Coding_Standards.md](#), [11_Build_y_Versioning_Guide.md](#), [13_Trazabilidad_y_Control_de_Cambios.xlsx](#), [26_Privacy_Data_and_Telemetry_Plan.md](#).

137. Credencial comprometida

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Credencial comprometida**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Credencial comprometida: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

138. Repositorio comprometido

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Repositorio comprometido**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.

7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Repositorio comprometido: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

139. Paquete malicioso

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Paquete malicioso**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Paquete malicioso: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: [09_CSharp_Coding_Standards.md](#), [11_Build_y_Versioning_Guide.md](#), [13_Trazabilidad_y_Control_de_Cambios.xlsx](#), [23_Legal_Credits_and_Licenses_Register.xlsx](#).

140. Build alterada

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Build alterada**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Build alterada: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `24_Steam_Publishing_Plan.md`.

141. Malware o ransomware

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Malware o ransomware**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.

7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Malware o ransomware: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

142. Pérdida de dispositivo

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Pérdida de dispositivo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Pérdida de dispositivo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

143. Publicación accidental

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Publicación accidental**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Publicación accidental: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

144. Secretos en commit

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Secretos en commit**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados. El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.

6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.
8. Detener propagación y acceso sin borrar evidencia.
9. Revocar/rotar credenciales y revisar sesiones activas.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.
- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Secretos en commit: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.
- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

145. Save exploit crítico

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Save exploit crítico**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La persistencia usa `Application.persistentDataPath`, envelope con SHA-256, primario, `.bak`, `.tmp` y `.recovery`, y `schemaVersion = 2`. El checksum detecta corrupción accidental, pero no autentica contra un atacante local. El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. Detener propagación y acceso sin borrar evidencia.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Save exploit crítico: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

146. Denegación de servicio futura

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Denegación de servicio futura**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece **NOT OPEN**.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.

2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Denegación de servicio futura: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

147. Abuso de comunidad

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Abuso de comunidad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Abuso de comunidad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `25_Post_Launch_and_Live_Operations_Plan.md`.

148. Plan de crisis

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Plan de crisis**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es de una sola persona y carece de equipo permanente de crisis; la simplicidad operativa es un requisito de seguridad.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Mantener una hoja de una página con prioridades, contactos y decisiones irreversibles prohibidas.
6. Pausar releases y preservar cuentas/artefactos.
7. Solicitar apoyo profesional cuando el incidente supere capacidad técnica, legal o emocional.

Evidencia y criterio de gate

- Crisis log y decisiones Go/No-Go.
- Contactos verificados y simulacro.

Criterio PASS de Plan de crisis: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Tomar decisiones destructivas bajo presión.
- No comunicar por sobrecarga del único owner.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

149. Business continuity

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Business continuity**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Business continuity: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

150. RTO y RPO

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **RTO y RPO**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de RTO y RPO: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

151. Single-person dependency

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Single-person dependency**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada. Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.

5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.
9. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.
- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Single-person dependency: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.
- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: [09_CSharp_Coding_Standards.md](#), [11_Build_y_Versioning_Guide.md](#), [13_Trazabilidad_y_Control_de_Cambios.xlsx](#), [23_Legal_Credits_and_Licenses_Register.xlsx](#).

152. Recuperación de cuentas

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Recuperación de cuentas**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario. Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.
9. Preservar copia antes de modificar el sistema afectado.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.
- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Recuperación de cuentas: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.
- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.
- Convertir la documentación en cumplimiento de papel.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

153. Contactos de emergencia

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Contactos de emergencia**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Contactos de emergencia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

154. Simulacros

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Simulacros**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera **PLANNED** hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Simulacros: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

155. Métricas de seguridad

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Métricas de seguridad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Usar IDs estables, owner, target, evidencia y estado.
6. Cerrar deuda solo con prueba; una aceptación necesita vencimiento y riesgo residual.
7. Revisar gates al cambiar alcance, proveedor o build.

Evidencia y criterio de gate

- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Métricas de seguridad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Convertir la documentación en cumplimiento de papel.
- Mantener controles obsoletos sin ejecutar.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

156. Security debt

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Security debt**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.

- 4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
- 5. Usar IDs estables, owner, target, evidencia y estado.
- 6. Cerrar deuda solo con prueba; una aceptación necesita vencimiento y riesgo residual.
- 7. Revisar gates al cambiar alcance, proveedor o build.

Evidencia y criterio de gate

- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Security debt: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Convertir la documentación en cumplimiento de papel.
- Mantener controles obsoletos sin ejecutar.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md , 11_Build_y_Versioning_Guide.md , 13_Trazabilidad_y_Control_de_Cambios.xlsx .

157. Work packages

WP	Alcance	Objetivo	Evidencia de cierre
SEC-WP-01	Inventario de activos, cuentas y trust boundaries	S17	Asset/account register + owner
SEC-WP-02	MFA, password manager y recovery	S17	Captura/registro sin secretos
SEC-WP-03	Backup 3-2-1 y restore drill	S17/H6	Restore report
SEC-WP-04	SECURITY.md y canal privado	Pre-public	Published policy

WP	Alcance	Objetivo	Evidencia de cierre
SEC-WP-05	Secret scan de árbol e historial	S17	Scan report + remediation
SEC-WP-06	Branch protection y revisión compensatoria	S17	Repository settings
SEC-WP-07	SBOM y dependency advisory process	S17/H6	SBOM + triage log
SEC-WP-08	Auditoría UnityConnect/ProjectSettings	S17	Settings decision + network observation if needed
SEC-WP-09	Tests de paths, borrado y save tampering	S17	EditMode/PlayMode evidence
SEC-WP-10	Build allowlist y exclusión TestLab	H6	Build contents report
SEC-WP-11	Signing/reputation decision Windows	Pre-Steam	ADR and certificate plan
SEC-WP-12	Steamworks IAM y SteamPipe secrets	Pre-Steam	Roles, MFA, upload runbook
SEC-WP-13	Incident register y templates	S17	Dry run
SEC-WP-14	Tabletop: leaked secret	S17	Exercise report
SEC-WP-15	Tabletop: bad build/rollback	H6	Exercise report
SEC-WP-16	Malware scan y clean-machine campaign	H6	Scanner + clean install evidence
SEC-WP-17	Vulnerability monitoring and patch SLA	Ongoing	Advisory log
SEC-WP-18	External security review trigger	Pre-release	Scope/decision
SEC-WP-19	Emergency continuity packet	H6	Custodied recovery instructions
SEC-WP-20	Final security Go/No-Go	H6/Steam	Signed decision

Cada WP tiene owner, dependencias, riesgo, coste estimado y criterio de rollback. Un WP no se cierra con “configurado”: debe conservar resultado reproducible y reabrirse cuando cambie el activo o proveedor.

158. Gates

Gate de desarrollo seguro

- árbol e historial sin secretos activos;
- dependencias inventariadas y lockfile revisado;
- tests afectados en PASS;
- ProjectSettings/Packages diffs aprobados;

- operaciones destructivas limitadas y probadas.

Gate H6

- cero SEC-0/SEC-1 abiertos;
- restore drill de repositorio y save;
- build limpia sin TestLab, tools, secrets o assets **NOT OPEN**;
- secret/dependency/malware scans revisados;
- rollback probado;
- incident templates y contactos listos;
- riesgo residual firmado.

Gate Steamworks

- cuenta individual, MFA, roles mínimos y recovery;
- VDF/secrets fuera de Git;
- upload desde workstation confiable;
- manifests/branches trazados;
- previous stable y revocación disponibles.

Gate de sistema online

Backend, Cloud, telemetría, Workshop o cuentas requieren threat model, privacidad, authn/authz, abuse controls, observabilidad, incident response, proveedor y shutdown plan. Hasta entonces: **NOT OPEN**.

159. Checklist pre-commit

- ☐ Diff limitado y entendible
- ☐ No secretos, tokens, VDF sensibles ni datos personales
- ☐ No binarios inesperados
- ☐ Tests targeted ejecutados
- ☐ Manifest/lock revisado si cambia

- ☐ Paths y borrados revisados
- ☐ Commit message y issue/ADR enlazados
- ☐ Working tree limpio tras commit

La checklist se adjunta al commit/build/release correspondiente. Los elementos no aplicables deben indicar motivo y aprobación; no se dejan en blanco.

160. Checklist pre-build

- ☐ Commit y versión identificados
- ☐ Secret scan
- ☐ Dependency/SBOM review
- ☐ Suites y targeted tests
- ☐ Lista de escenas aprobada
- ☐ TestLab y tools excluidos
- ☐ Development Build/script debugging según objetivo
- ☐ Artefactos previos eliminados o aislados
- ☐ Build en ruta controlada
- ☐ Checksum y log archivados
- ☐ Malware scan
- ☐ Ejecución como usuario estándar

La checklist se adjunta al commit/build/release correspondiente. Los elementos no aplicables deben indicar motivo y aprobación; no se dejan en blanco.

161. Checklist pre-release

- ☐ Candidata congelada
- ☐ Build/depot coincide con artefacto probado

- ☐ StoreInitial integrada
- ☐ Golden Path y siete días
- ☐ Install/update/uninstall
- ☐ Save migration/recovery
- ☐ Previous stable y rollback
- ☐ Créditos/notices/licencias
- ☐ Localización y privacidad
- ☐ MFA/cuentas/roles
- ☐ SEC-0/SEC-1 = 0
- ☐ Known issues y soporte
- ☐ Go/No-Go firmado

La checklist se adjunta al commit/build/release correspondiente. Los elementos no aplicables deben indicar motivo y aprobación; no se dejan en blanco.

162. Plantilla de incidente

```
Incident ID:  
Título:  
Fecha/hora UTC de detección:  
Reportado por / canal:  
Incident Commander:  
Severidad provisional / final:  
Activos y versiones afectados:  
Indicadores y evidencia preservada:  
Impacto confirmado / potencial:  
Datos personales implicados:  
Contención aplicada:  
Credenciales revocadas/rotadas:  
Causa raíz:  
Erradicación:  
Recuperación y tests:  
Comunicación interna/externa:  
Proveedor/Valve/autoridad contactados:  
Timeline UTC:  
Riesgo residual:  
Acciones y owners:
```

Fecha de cierre:
Postmortem:

163. Plantilla de vulnerability report

Vulnerability ID:
Versión/build/commit:
Componente:
Descripción técnica:
Precondiciones:
Pasos de reproducción o PoC seguro:
Impacto:
Alcance:
Datos o secretos expuestos:
Mitigación temporal:
Divulgación previa/pública:
Contacto del reportante:
Permiso para acreditar:
Adjuntos y hashes:

No se exige al reportante ejecutar acciones destructivas, acceder a datos de terceros ni publicar la vulnerabilidad. La respuesta inicial confirma recepción y proporciona un identificador; no promete recompensa o fecha si no existe programa aprobado.

164. Plantilla de risk acceptance

Risk ID:
Activo/sistema:
Amenaza y escenario:
Probabilidad / impacto / exposición:
Control faltante:
Motivo para no corregir ahora:
Compensating controls:
Versiones/builds autorizadas:
Owner:
Fecha de expiración:
Condiciones de reapertura:
Aprobador:
Evidencia:

No se acepta de forma indefinida un riesgo de secreto activo, cuenta de publicación comprometida, build manipulada, pérdida generalizada de save, malware, incumplimiento legal o exposición de datos personales.

165. RACI

Actividad	Owner/IC	Desarrollo	QA	Legal/Privacidad	Plataforma/Proveedor	Revisor externo
Inventario y controles	A/R	R	C	C	I	C
Patch de dependencia	A	R	R	C	C	C
SEC-0/SEC-1	A/R	R	R	C	C	C
Brecha de datos	A	C	C	R	C	C
Steam credential incident	A/R	C	C	C	R/C	C
Release Go/No-Go	A	R	R	C	C	C
Comunicación pública	A	C	C	C	C	C

En el estado actual, el owner acumula funciones. A/R no elimina la necesidad de revisión compensatoria para una publicación pública o incidente severo.

166. Definition of Ready

Un trabajo con impacto de seguridad está Ready cuando:

- activo y owner están identificados;
- amenaza, trust boundary y datos están descritos;
- dependencias y proveedor son conocidos;
- existe plan de test, rollback y evidencia;
- secretos/roles se han diseñado sin valores embebidos;
- legal, privacidad y licencias han sido consultados cuando aplica;
- no abre silenciosamente un sistema NOT OPEN;
- el riesgo de no hacerlo también está registrado.

167. Definition of Done

Un control o cambio está Done cuando:

- implementación y configuración están versionadas;
- revisión y tests pasan en el entorno objetivo;
- el resultado negativo también se ha probado cuando es relevante;
- no existen secretos o datos inesperados en código, logs y build;
- documentación, threat model, SBOM y notices están actualizados;
- recuperación/rollback se ha ejecutado, no solo descrito;
- findings se han cerrado o aceptado con vencimiento;
- la evidencia está archivada y enlazada;
- el gate afectado se ha actualizado manualmente.

168. Riesgos abiertos

Risk ID	Riesgo	Nivel inicial	Tratamiento mínimo
SEC-RISK-001	Single-person dependency	High	Backup, emergency packet and external review
SEC-RISK-002	No restore drill	Critical	3-2-1 backup and quarterly restore
SEC-RISK-003	No remote secret/history scan evidence	High	Scan history and enable prevention
SEC-RISK-004	No dependency advisory process	High	SBOM and periodic CVE/advisory review
SEC-RISK-005	TestLab included in build profile	High	Release allowlist and build content test
SEC-RISK-006	StoreInitial absent from build profile	High	Controlled migration and external build campaign
SEC-RISK-007	Unity diagnostics settings ambiguous	Medium	Audit settings and observed traffic
SEC-RISK-008	No code-signing decision	Medium	Authenticode/reputation ADR before public release
SEC-RISK-009	No SECURITY.md/private channel	High	Publish before external testing
SEC-RISK-010	No Steamworks IAM yet designed operationally	High	MFA/roles/recovery before onboarding
SEC-RISK-011	File deletion paths insufficiently security-tested	Medium	Path boundary and destructive-operation tests
SEC-RISK-012	Save checksum may be mistaken for anti-tamper	Medium	Document limits and validate invariants

Risk ID	Riesgo	Nivel inicial	Tratamiento mínimo
SEC-RISK-013	No malware scan/clean machine evidence	High	Candidate scan and clean install
SEC-RISK-014	No tabletop exercises	Medium	Secret leak and bad build exercises
SEC-RISK-015	No external review trigger defined	Medium	Trigger by online systems/public release
SEC-RISK-016	Package LICENSE/NOTICE still open	High	Coordinate with legal register
SEC-RISK-017	No secure support channel	Medium	Create with privacy/retention controls
SEC-RISK-018	Build reproducibility not proven across clean environment	High	Clean workstation/VM rebuild
SEC-RISK-019	Backups may share same failure domain	High	Offline/offsite copy
SEC-RISK-020	Future AI/tool uploads may expose confidential data	Medium	Tool data policy and sanitization

Los niveles son de planificación, no una evaluación legal definitiva. Mientras no se genere el futuro [33_Project_Risk_Register_and_Business_Continuity_Plan.xlsx](#), este capítulo y [13_Trazabilidad_y_Control_de_Cambios.xlsx](#) conservan el contexto y seguimiento operativo de seguridad.

169. Glosario

Término	Definición operativa
Activo	Elemento con valor para el proyecto.
Advisory	Aviso sobre vulnerabilidad o riesgo de un componente.
Build provenance	Evidencia de origen y proceso de una build.
Causa raíz	Condición que permitió el incidente y cuya corrección previene recurrencia.
Contención	Acción para limitar impacto inmediato.
CVE	Identificador público de una vulnerabilidad conocida.
Evento	Observación que puede o no ser incidente.
Incidente	Evento que compromete o amenaza confidencialidad, integridad, disponibilidad o autenticidad.
IoC	Indicador de compromiso.
MFA	Autenticación con más de un factor.
RPO	Pérdida máxima de datos tolerada medida en tiempo.

Término	Definición operativa
RTO	Tiempo objetivo para recuperar un servicio/activo.
SBOM	Inventario de componentes de software y versiones.
Secret	Credencial que concede acceso o firma.
SAST	Análisis estático de seguridad.
SCA	Análisis de componentes y dependencias.
Threat model	Modelo de activos, actores, límites y amenazas.
Trust boundary	Punto donde cambia el nivel de confianza.
Vulnerability disclosure	Proceso coordinado de recepción, corrección y comunicación.
Zero-day	Vulnerabilidad sin corrección disponible o desconocida previamente.

170. Fuentes oficiales

Fuentes verificadas el **2026-07-01**. Deben revalidarse antes de una decisión externa, especialmente si cambian GitHub, Steamworks, Unity, NIST u OWASP.

ID	Fuente	URL	Uso en el plan
NIST-CSF-2.0	NIST Cybersecurity Framework 2.0	https://www.nist.gov/cyberframework	Gobernanza y gestión del riesgo mediante Govern, Identify, Protect, Detect, Respond y Recover.
NIST-SSDF-1.1	NIST SP 800-218 — Secure Software Development Framework 1.1	https://src.nist.gov/pubs/sp/800/218/final	Prácticas de desarrollo seguro integrables en el ciclo de vida.
NIST-IR-3	NIST SP 800-61 Rev. 3 — Incident Response Recommendations	https://src.nist.gov/pubs/sp/800/61/r3/final	Perfil CSF 2.0 de preparación, detección, respuesta y recuperación; publicado en abril de 2025.
CISA-SBD	CISA Secure by Design	https://www.cisa.gov/securebydesign	Responsabilidad del productor, defaults seguros y reducción del coste de seguridad para el usuario.
OWASP-A03-2025	OWASP Top 10:2025 — A03 Software Supply Chain Failures	https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/	Inventario de dependencias directas/transitivas, fuentes confiables, hardening y change management.

ID	Fuente	URL	Uso en el plan
GITHUB-SECRETS	GitHub Docs — Secret scanning	https://docs.github.com/en/code-security/concepts/secret-security/secret-scanning	Detección y prevención de secretos expuestos, incluyendo push protection cuando esté disponible.
GITHUB-BRANCHES	GitHub Docs — About protected branches	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches	Protección de ramas, checks y restricciones de actualización.
GITHUB-SECURITY-POLICY	GitHub Docs — Adding a security policy	https://docs.github.com/en/code-security/how-tos/report-and-fix-vulnerabilities/configure-vulnerability-reporting/add-security-policy	Uso de SECURITY.md para versiones soportadas y canal privado de reporte.
GITHUB-DEPENDABOT	GitHub Docs — Dependabot alerts	https://docs.github.com/en/code-security/concepts/supply-chain-security/dependabot-alerts	Alertas sobre dependencias vulnerables y seguimiento del riesgo de supply chain.
STEAM-USERS	Steamworks — Managing Your Steamworks Account	https://partner.steamgames.com/doc/gettingstarted/managing_users	Gestión de usuarios, grupos y permisos del partner.
STEAM-UPLOAD	Steamworks — Uploading to Steam	https://partner.steamgames.com/doc/sdk/uploading	SteamPipe, scripts de build, depots, credenciales y publicación de builds.
UNITY-UPM	Unity Manual — Switch to another version of a UPM package	https://docs.unity3d.com/Manual/upm-ui-update.html	Cambios de versión de paquetes y necesidad de probar compatibilidad.

Las referencias se usan como guía proporcional. No se afirma cumplimiento certificado ni se copian requisitos de entornos empresariales que no aportan valor al proyecto.

171. Inventario histórico

Se han conservado **234 fuentes históricas u operativas relevantes**. El filtro es deliberadamente amplio e incluye builds, QA, ADR, handoffs, legal, paquetes, Steam, persistencia y auditorías; una fuente listada puede ser contextual y no una autoridad de seguridad autónoma.

Baseline	Tip o	Ruta	Bytes	SHA-256
0.3	.pdf	Documentation/00_Official_Baseline/v0.3/00_Project_Governance/Cartridge_And_Cloud_Package_Manifest_v0.1.pdf	40833	4aed1b174d2405952bd79800726f58314e2bf96842da2b233cf005db84689f83
0.3	.pdf	Documentation/00_Official_Baseline/v0.3/02_Technical/Cartridge_And_Cloud_Build_Versioning_Guide_v0.3.pdf	25562	81e38d9861f5caee0d4448c0aff214e0ffc922e82e2491f8b37c7f37719bd97a
0.3	.pdf	Documentation/00_Official_Baseline/v0.3/02_Technical/Cartridge_And_Cloud_CSharp_Coding_Standards_v0.3.pdf	27744	7728a56bdb4c65c4a74cd02791293449dd2277d5a544f038744476c3c0b73c43
0.3	.pdf	Documentation/00_Official_Baseline/v0.3/02_Technical/Cartridge_And_Cloud_Unity_Project_Setup_Guide_v0.3.pdf	28287	c0725431f52d44b931bc58773ebd56fae72d00dca0ff456c41167a1a7705e537

Baseline	Tip o	Ruta	Bytes	SHA-256
0.3	.pdf	Documentation/00_Official_Baseline/v0.3/04_QA_Production/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.3.pdf	28949	2b48b54738598dbaa9b461d7cad6f4a6a2f70bb8589df55e7242ec7c9e40e6e
0.3	.pdf	Documentation/00_Official_Baseline/v0.3/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Plan_v0.3.pdf	393009	2d71d4dd720598429744bc6cf959162a8915301ad69171bc6816168e47f75709
0.3	.pdf	Documentation/00_Official_Baseline/v0.3/05_Business_Release/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.3.pdf	22534	e358622a237b57cdcd65fb380460d70eafa0ef1085b43bf5e8fb7dff3ee734f0
0.3	.pdf	Documentation/00_Official_Baseline/v0.3/05_Business_Release/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.3.pdf	231402	10ff2852e8621c210bb578f63181382665ccbedeba3d62bfdce7eb4cb4e53472
0.3	.md	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Build_Versioning_Guide_v0.3.md	3678	fcc0988f428b45303c4cc2f5a681cbed9970475667bb6a37fa97a36529dc3705
0.3	.md	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_CSharp_Coding_Standards_v0.3.md	4466	2a948ae7f681636e28761e91da7828ecbe44c15e5e0b02efa2cb8e0a6873278f
0.3	.md	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.3.md	3972	ffc5bc6af65b27ede021acfdedef2dbc5f20f680560e0be1a872586ab146a2f101
0.3	.md	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Package_Manifest_v0.1.md	6551	58b7f326720277d2b490784a8091cfcf34f3182ac4fff57796e70d7817d9a4bc1
0.3	.md	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.3.md	5738	93b345037c1356c7e176a7377bd66a168aa356366aa3c4e16062b09fcb46ee60
0.3	.md	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_QA_Testing_Plan_v0.3.md	9293	517a90d50868dd91010be4eaaca335326b63e8d370613c82a5b5664753ab3891
0.3	.md	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.3.md	3923	1a004be75991bdcbbaf457a57da123e3297fce15a38c591e3f589e11e277dbd5
0.3	.md	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Unity_Project_Setup_Guide_v0.3.md	4564	0a6f42c71d9d22cee8aa12796842fd5391c3570f675f7361f1932418b74e1e55
0.3	.txt	Documentation/00_Official_Baseline/v0.3/CHECKSUMS_SHA256.txt	7808	8312237b079e9a3f92dadaf41711f788caf08fb641aca3b2676ca5427accedc3
0.3	.md	Documentation/00_Official_Baseline/v0.3/PACKAGE_MANIFEST.md	6098	7fb1bdbd0b11a372ccf4ad87efa6f88326188a21a8f4760680b0e49cd928783
0.4	.pdf	Documentation/00_Official_Baseline/v0.4/00_Project_Governance/Cartridge_And_Cloud_Package_Manifest_v0.2.pdf	36002	2fb3d34140d531bb28015e95ebed32e69d5e38f40fd2e591f99af99304035b5e
0.4	.pdf	Documentation/00_Official_Baseline/v0.4/02_Technical/Cartridge_And_Cloud_Build_Versioning_Guide_v0.4.pdf	46424	b26821c7646a0648c9bca240688bc7e3f82edd0c5ff853ac115e88ee3a742c4c
0.4	.pdf	Documentation/00_Official_Baseline/v0.4/02_Technical/Cartridge_And_Cloud_CSharp_Coding_Standards_v0.3.pdf	44429	996a4d5fe70a5a60c8e0194dfd167f527db03d9965cfdae83f6b8352447fec64
0.4	.pdf	Documentation/00_Official_Baseline/v0.4/02_Technical/Cartridge_And_Cloud_Unity_Project_Setup_Guide_v0.4.pdf	50446	c12af29fd274e9ae5e297364a7c48f65b78de03445aa666fcd8556339a7fc8b3
0.4	.pdf	Documentation/00_Official_Baseline/v0.4/04_QA_Production/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.4.pdf	45968	a633ddefdface673e33411b00fd52a84d6984fecf5a49586352d2efb95ef4951
0.4	.pdf	Documentation/00_Official_Baseline/v0.4/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Plan_v0.4.pdf	59968	047a7bdd0a0b1ed6f9dec50aacc429bf1f18a2ba874fca0a28a270cc61bbfa45
0.4	.pdf	Documentation/00_Official_Baseline/v0.4/05_Business_Release/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.3.pdf	42670	e4105813b235d24b75edf539eae505342f58f1f26a4b087e870dcba9d63f558
0.4	.pdf	Documentation/00_Official_Baseline/v0.4/05_Business_Release/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.3.pdf	43916	b052a99f72f3b531b75b5950e5459fc6bd2a62c516be7dc1ad5e53d3a4684ec3
0.4	.md	Documentation/00_Official_Baseline/v0.4/06_Development_Records/Governance/Project_Foundation_Release_Record.md	466	f789933c54973d329e078dd5a561b43f174333653a36aca00ce5254daaac0aab

Baseline	Tip o	Ruta	Bytes	SHA-256
0.4	.md	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Build_Versioning_Guide_v0.4.md	4723	6ac0c7072b5ac4b1844d82daffe1ec5f241dba369145e5c62152e290b8309acd
0.4	.md	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_CSharp_Coding_Standards_v0.3.md	4571	d49fa93c9948fe7c66d049193e866b80a8aa58dea240d666941ceca171e38565
0.4	.md	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.3.md	4077	1a0e921f139b73179fe734eec452dcc3aa8c43860309390fe41ea1fb588fe81f
0.4	.md	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Package_Manifest_v0.2.md	2456	ccf1ff8da3ecfb69e1ee93a902e1d13d706d5e7ada9ac4a185515c8f93cd2b7a
0.4	.md	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.4.md	6571	e9976daa630e8bc1460831eea0c3d280d24f38d3907ef3dd97baec6e35661404
0.4	.md	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_QA_Testing_Plan_v0.4.md	10116	623c83bd390d3fb1e7cb09e554f6a29f63db662c7ca3e296c6593c65969a724
0.4	.md	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.3.md	4028	56979c2830951e4a82a7d01793cebe9a930df4a773245de2b7489d0e73cb0777
0.4	.md	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Unity_Project_Setup_Guide_v0.4.md	5564	0feb637a512b120a1515127467a5a9af0e97758c5a2f93627b6a4f3165b63cf8
0.4	.txt	Documentation/00_Official_Baseline/v0.4/CHECKSUMS_SHA256.txt	9591	f07be7f38ffc3ed61499d44aeb0e08d3341f602758a090b3ce95e3eec5109464
0.4	.md	Documentation/00_Official_Baseline/v0.4/PACKAGE_MANIFEST.md	2456	ccf1ff8da3ecfb69e1ee93a902e1d13d706d5e7ada9ac4a185515c8f93cd2b7a
0.5	.pdf	Documentation/00_Official_Baseline/v0.5/00_Project_Governance/Cartridge_And_Cloud_Package_Manifest_v0.3.pdf	16782	d74ae4fb1a2590813315e14d668ce2eca7a22081a7acdafbb5a55e07fb7a4f87
0.5	.pdf	Documentation/00_Official_Baseline/v0.5/02_Technical/Cartridge_And_Cloud_Build_Versioning_Guide_v0.5.pdf	18588	34fc32adf9c6383d0bf5b79ed446cd4dd00c9518cbd67c6e9d7cd5623b236347
0.5	.pdf	Documentation/00_Official_Baseline/v0.5/02_Technical/Cartridge_And_Cloud_CSharp_Coding_Standards_v0.4.pdf	20856	dbb8bd5fd565a2c4078c26f56d1d1474daaa44ba98920fcf3bfc72f4a4081632
0.5	.pdf	Documentation/00_Official_Baseline/v0.5/02_Technical/Cartridge_And_Cloud_Unity_Project_Setup_Guide_v0.5.pdf	20165	5892ba151a6fce83012f252b41260b2bac73809cb27559ab4f3fcf03173e89b0
0.5	.pdf	Documentation/00_Official_Baseline/v0.5/04_QA_Production/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.5.pdf	21719	d75c34e0085a41c5a875fd17cf8fe3f7ea3f6b6ba6e496246a2cf93e06427b93
0.5	.pdf	Documentation/00_Official_Baseline/v0.5/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Plan_v0.5.pdf	20203	0763d0d411746199eaa762acc995f591393eb39a1c8d88511c4dec488e56e74e
0.5	.pdf	Documentation/00_Official_Baseline/v0.5/05_Business_Release/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.4.pdf	15779	d8fa9f63a454facc7e8d4e3fa8e50d1a95f0bb7a8f8bef06211e9d12accb5343
0.5	.pdf	Documentation/00_Official_Baseline/v0.5/05_Business_Release/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.4.pdf	16309	f0382da1359aa1339758e067dc58c2d921c46ca37aef870c74ecee0ef320a3b2
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Handoff/CURRENT_PROJECT_HANDOFF.md	2275	9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_01/Governance/ADR-0010_Phase_Level_Build_Evidence_and_Release_Policy.md	2861	ad328a9eabc2302b0688591df010650b693d9423f0f485eb1000e24d50475184
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_01/QA/S1_Acceptance_Matrix.md	1455	a5a4e0a192fe33559c582efd9ee4d53170e26af9e07db63aae1e22b1bf5c66f49
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_01/QA/S1_QA_Execution_Record.md	1065	f9ae3d8cb7bd112af8d518e93ccfe03fbce2309d555f3130b0325427ef68daad
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_02/Governance/ADR-0011_Minimal_Versioned_Save_Skeleton.md	2505	4048b071ec41b0d5caf4beb0d5a4ff7547e409f566a403d0e9331a604de076dd

Baseline	Tip o	Ruta	Bytes	SHA-256
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_02/QA/S2_Acceptance_Matrix.md	993	1c184fd5c44de6051e83614676a1571d4d49e7cd8b4d68c8ef85167b64e2692f
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_02/QA/S2_QA_Execution_Record.md	1827	70ad9d5ab6b32071b69fcbcb35e7f76889b95b4d84ec6c6e174ab2d793ccea4f
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/Governance/ADR-0012_Scene_Driven_Input_Contexts.md	1159	13c16537cc90fb0cdf3e74d588a585635c8b7b998433a1ed52501aad1d89af0f
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/Governance/ADR-0013_Direct_Planar_Click_To_Move_Foundation.md	1449	24601e21279c905274e2c7092c1504eae9f72be23477d15741fab33d79971059
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/Governance/ADR-0014_Orbit_Zoom_Follow_Camera.md	1331	c7bcb3d83c5544605bac741e625f1a441f5ba10047e204e4ce6dbd9eb7468a77
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/Governance/ADR-0015_Project_Input_Actions_And_Context_Routing.md	1653	b665c0f33852bfb4ae7d0de42fac1636487181e19c28593c1d728d412e21954e
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/QA/S3.4_Build_Execution_Record.md	1303	bdbcbce899c627a7abf1f302f7db42ce29de1f0809c8373a454e3bc070984ab7d
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/QA/S3_Acceptance_Matrix.md	1643	dcd3a753ec03bf462ed61fee96f551716970f05a54702fcc5ebdc271bda15b7
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/QA/S3_QA_Execution_Record.md	2573	c36caf9cccc1d762ab9f201d7eaedf2e07ae88f1787d7ed0df6bf13305850116
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/Governance/ADR-0016_Grid_Coordinate_And_Footprint_Foundation.md	1157	84ffb0f806ec18937785dc30f5410a0be471ea75deeeb3d23021657ff513d622
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/Governance/ADR-0017_Placement_Preview_And_Rotation.md	1125	996ccbe4400bc1971fea44b1d1e89b7a0cbe30437fa660283af16e79b8f18965
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/Governance/ADR-0018_Atomic_Occupancy_And_Placement_Mode.md	1439	3cdafe143aedd7a015756ec256b6c92be2aaaa918c33499421589d3fb51177e1
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/Governance/ADR-0019_Sprint_04_Final_Integration_And_Build_Gate.md	1324	f78c4971ab6969d8d3f302b93ceb5dbf30f2d553f433122bd8d24eb12fa7dff3
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/QA/S4.4_Build_Execution_Record.md	1164	6f662c5fe7cce7a3080584798ccf51c720d4ade1dc50b1be2f8c4c107da529d
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/QA/S4_Acceptance_Matrix.md	2209	30a86d51980351f806ef30aa6187ce90857bde474c163d47a6a3396b60522f89
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/QA/S4_QA_Execution_Record.md	2537	0c5ec51188fd94404436c7093e01ed0a0d0d75fc466da104e777fd7494aa0603
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/Governance/ADR-0020_Initial_Store_Shell_Dimensions_And_Entrance.md	870	af706a1741b657833a36805700e10d25e2b66264438b488c1a8983e883799e3d
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/Governance/ADR-0021_Logical_Store_Access_Validation.md	1131	079eab3c8041ef624f17ba35976f8ac46627ed9ff192a910e1f0f4f63fee5fe7
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/Governance/ADR-0022_Store_Placement_And_Access_Integration.md	1085	f180b5eac43281274d001667b8fc49f39febfbacfeb78fec7d686b9e07546524
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/Governance/ADR-0023_Sprint_05_Final_Integration_And_Build_Gate.md	972	86eaea910888d35b0bb89c6a30aef788d767f38b95283339487890c29844f476
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/QA/S5.4_Build_Execution_Record.md	1347	83700192228af99fb4487bdf4cdd1332d13a194be7cd5fb197278013952f8fe1
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/QA/S5_Acceptance_Matrix.md	2498	3c265e9509b133370788c1ea024e4de481f773b526a524f2ec880d60abe053b6
0.5	.md	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/QA/S5_QA_Execution_Record.md	2607	28e52cfc1189efc63cafabf899b0b94924bf2fb3edb92a43973797fcb77616d4

Baseline	Tip o	Ruta	Bytes	SHA-256
0.5	.md	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Build_Versioning_Guide_v0.5.md	2384	3df0fc5321a57c528f6386bcd922c2e930ba194736af94f064262c371bb63be6
0.5	.md	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_CSharp_Coding_Standards_v0.4.md	2940	e83ba0009eda604345b2939a8cddf7f8d1a4b8b3ab2445bfcf79996889844e6d
0.5	.md	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.4.md	2001	18f78b69a56edee6f8783d32859d6e231d061422d24abd6feefdbf06fe68567b
0.5	.md	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Package_Manifest_v0.3.md	1783	7ae705ca39b1e39345748c208af4274cbb9eaf887a249e5666bceefadf9d0d2
0.5	.md	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.5.md	3654	d0623a03b2ff75fa95ecd6bbaa5fcbaf8aada31fc2a6b4a569caa50f0e285
0.5	.md	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_QA_Testing_Plan_v0.5.md	2853	c19ab67f26aae4b44fd5c6d431fb163ddbfcfecbccc0ca68518826040b534f1c
0.5	.md	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.4.md	2025	19664d4092ea89898c5bc37483a0b1aba6f473abbfa02c2794df8f2dd482058d
0.5	.md	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Unity_Project_Setup_Guide_v0.5.md	2491	cb4a9ea65fc1c8b95df59019c3bf6887cf220606980d221904494dfb057cf6ca
0.5	.txt	Documentation/00_Official_Baseline/v0.5/CHECKSUMS_SHA256.txt	20623	14fd238f4a1c84ac2414fb856bb69b84b18f008e1c23de6e08b6122e9e822b46
0.5	.md	Documentation/00_Official_Baseline/v0.5/CURRENT_PROJECT_HANDOFF.md	2275	9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396
0.5	.md	Documentation/00_Official_Baseline/v0.5/PACKAGE_MANIFEST.md	814	8bb540cf6f6eb6e6f079cf55786ab3b53f4ec042bb1dd01147edb03222f306c5
0.6	.pdf	Documentation/00_Official_Baseline/v0.6/00_Project_Governance/Cartridge_And_Cloud_Package_Manifest_v0.4.pdf	31216	e2b87252fcc868ccf8c9bd6b7e9737260653ed2ae07d2eb86394cb503d64f48c
0.6	.pdf	Documentation/00_Official_Baseline/v0.6/02_Technical/Cartridge_And_Cloud_Build_Versioning_Guide_v0.6.pdf	37901	758372f65f3e817c8cc192d557cac69f2ecb2b22d0705e8feeac9e5034cf0a02
0.6	.pdf	Documentation/00_Official_Baseline/v0.6/02_Technical/Cartridge_And_Cloud_CSharp_Coding_Standards_v0.5.pdf	38900	bcbd99d67deb5d0d1f6a3e90a8fb6058cbd4798b1b011ff6ad4e0980642383ad
0.6	.pdf	Documentation/00_Official_Baseline/v0.6/02_Technical/Cartridge_And_Cloud_Unity_Project_Setup_Guide_v0.6.pdf	40504	415fcd4c16c68e395c49179492c10db67374b3fb1c28257ec41987b2735d7ba8
0.6	.pdf	Documentation/00_Official_Baseline/v0.6/04_QA_Production/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.6.pdf	37116	5e1a0fea9e45f7af252162baba7544778134216c0f9c839aaabdd48687b3c370
0.6	.pdf	Documentation/00_Official_Baseline/v0.6/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Plan_v0.6.pdf	38292	ff451a919545374e6bd22a44efd7f8e65d3e9001741a2ddcac58b4f339052c04
0.6	.pdf	Documentation/00_Official_Baseline/v0.6/05_Business_Release/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.5.pdf	28754	eee199263d089442dc5ea6228b10b562d7930163c707a7e90402e734bf94db51
0.6	.pdf	Documentation/00_Official_Baseline/v0.6/05_Business_Release/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.5.pdf	28734	a21ea9abe67f1d825731d25b8608c8f7b67e5e1b437356e0e1ee3d97870d7728
0.6	.md	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Governance/ADR-0035_StoreInitial_Manual_Scene_Authoring.md	847	0e69d8b664239477738f3dcce96eb5cfce2ff6e3db8817261f639ac4d64092b3
0.6	.md	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Handoff/CURRENT_PROJECT_HANDOFF.md	1241	7a297d84a36ecdd9e4296a631fc5acbff882218d12563312850e8f28b46cd87
0.6	.md	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Handoff/NEXT_CHAT_START_HERE.md	896	e0cca56b839dc2071363b310c06543178dbf978622990fc7f4be301e077fc754
0.6	.md	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_16/QA/S16_Acceptance_Matrix.md	572	4ad529aceeaf7e2ea87def0e0be2f8f3998d6218f6ac11a06b40f770e66c44fd

Baseline	Tip o	Ruta	Bytes	SHA-256
0.6	.md	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_16/QA/S16_QA_Execution_Record.md	328	40c6de3168db5dcf5c71afc166c0a3a2bf34f2dc3e71344c3cea6176d573766
0.6	.md	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Build_Versioning_Guide_v0.6.md	2462	8cd0f62e233cfbad2bed3a07122696e2ea81b4fd5280987cffb9a8a2a1e98d43
0.6	.md	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_CSharp_Coding_Standards_v0.5.md	2690	99f4cf1616d3c5e3d825ce713c6996928a09fa5c28c43802c4104336a28bfe57
0.6	.md	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.5.md	1951	610fc2ca1dd8ea1859fb779039e5de841610646d1e5601d316b2fe34410c3c81
0.6	.md	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Package_Manifest_v0.4.md	1910	89ae5a7d733d8d70f36b64cfe3653aed99149827155fcddec0cadf650fd38acb
0.6	.md	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.6.md	3191	f7b50c87cbc0faa2478f677f868e2e29dd1e74abfe74c1cbb30aed74ab7ac70a
0.6	.md	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_QA_Testing_Plan_v0.6.md	2637	7659a777a9aa78f083d8a6d577898bec89430cae108b079ee7137c769da7f069
0.6	.md	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.5.md	1866	f7840ee791e07430dd750500421dc6e262d21d5119e634634919247ff13cf8d0
0.6	.md	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Unity_Project_Setup_Guide_v0.6.md	2805	e0ba621f7cbd548da5ffa6484a1c2a0c91b820f845e5d410e318f01c34547497
0.6	.txt	Documentation/00_Official_Baseline/v0.6/CHECKSUMS_SHA256.txt	10464	129b78704f6af279a789ad01988ca19be78fcd30d3952202633cdf061b5e4b86
0.6	.md	Documentation/00_Official_Baseline/v0.6/CURRENT_PROJECT_HANDOFF.md	1241	7a297d84a36ecdd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87
0.6	.md	Documentation/00_Official_Baseline/v0.6/PACKAGE_MANIFEST.md	876	c25b2f638ce7814f8f2d9a153481efe6891a01b7e873f1a3251775ee51b922dd
working/current	.md	Documentation/10_Development_Records/ADR/ADR-0001-Unity-and-Package-Baseline.md	831	196d02854bf9a34c0e4c2acd7056cde24e0a23c74d345f900997fff9689f853
working/current	.md	Documentation/10_Development_Records/ADR/ADR-0002-Version-Control-and-Repository-Workflow.md	701	53e4ccc17554147c05af07525dbbe96401d99cc339774c075dfdb835edfb099
working/current	.md	Documentation/10_Development_Records/ADR/ADR-0003-Documentation-Baseline-and-Operational-Layer.md	808	18a0308055e403fabd0e9b7a3d60910d22f9ba4dd2a35317408c441e905c605f
working/current	.md	Documentation/10_Development_Records/ADR/ADR-0004-Assembly-Dependency-Boundaries.md	792	8eea405d40efbffd5f51ad2c945994ffd82165bb5fef57900518c94fc23ac800
working/current	.md	Documentation/10_Development_Records/ADR/ADR-0005-Scene-Baseline-and-Build-Order.md	1662	e83bbf7ee82448366cdd997c18b11a620016ba4ea70b4576f8b159930db8880
working/current	.md	Documentation/10_Development_Records/ADR/ADR-0006-Smoke-Test-Baseline-and-Execution-Policy.md	1527	e2b6c379f89de34cfbf3064b62dad675d85560a4a0117ebc634cd49d703ab97a
working/current	.md	Documentation/10_Development_Records/ADR/ADR-0007-Windows-Development-Build-Baseline.md	2251	e62888a6608c248e56868b73025bd7044c455528178984f410f53fa18e942641
working/current	.md	Documentation/10_Development_Records/ADR/ADR-0008-Sprint-0-Foundation-Closure-Policy.md	1341	76b9266ef045590959eef0187fff7fa2c4dcbfa7f6cf173159873e39cdaca757
working/current	.md	Documentation/10_Development_Records/Documentation_Audit_Sprint_00.md	1781	60c000c0ffc0c06a3ae184e4c845b5224b02a42c053448f0a5d84bc0cb222b64
working/current	.md	Documentation/10_Development_Records/Handoff/CURRENT_PROJECT_HANDOFF.md	1168	ea48666ee43dbf978bef364ee645cd3ff32f31e4e6dfbd96b9edfab316546028
working/current	.md	Documentation/10_Development_Records/Sprint_01/Governance/ADR-0009_Bootstrap_Owned_Scene_Flow.md	1410	15408cc4633c35efc6f8b622230184e1636f9140898e48811a2f3498a0e0e2b5

Baseline	Tip o	Ruta	Bytes	SHA-256
working/current	.md	Documentation/10_Development_Records/Sprint_01/Governance/ADR-0010_Phase_Level_Build_Evidence_and_Release_Policy.md	2861	ad328a9eabc2302b0688591df010650b693d9423f0f485eb1000e24d50475184
working/current	.md	Documentation/10_Development_Records/Sprint_01/QA/S1_Acceptance_Matrix.md	1455	a5a4e0a192fe33559c582efd9ee4d53170e26af9e07db63aae1e22b1f5c66f49
working/current	.md	Documentation/10_Development_Records/Sprint_01/QA/S1_QA_Execution_Record.md	1065	f9ae3d8cb7bd112af8d518e93ccfe03fbce2309d555f3130b0325427ef68daad
working/current	.md	Documentation/10_Development_Records/Sprint_02/Governance/ADR-0011_Minimal_Versioned_Save_Skeleton.md	2505	4048b071ec41b0d5caf4beb0d5a4ff7547e409f566a403d0e9331a604de076dd
working/current	.md	Documentation/10_Development_Records/Sprint_02/QA/S2_Acceptance_Matrix.md	993	1c184fd5c44de6051e83614676a1571d4d49e7cd8b4d68c8ef85167b64e2692f
working/current	.md	Documentation/10_Development_Records/Sprint_02/QA/S2_QA_Execution_Record.md	1827	70ad9d5ab6b32071b69fcbb35e7f76889b95b4d84ec6c6e174ab2d793ccea4f
working/current	.md	Documentation/10_Development_Records/Sprint_03/Governance/ADR-0012_Scene_Driven_Input_Contexts.md	1159	13c16537cc90fb0cdf3e74d588a585635c8b7b998433a1ed52501aad18d9af0f
working/current	.md	Documentation/10_Development_Records/Sprint_03/Governance/ADR-0013_Direct_Planar_Click_To_Move_Foundation.md	1449	24601e21279c905274e2c7092c1504eae9f72be23477d15741fab33d79971059
working/current	.md	Documentation/10_Development_Records/Sprint_03/Governance/ADR-0014_Orbit_Zoom_Follow_Camera.md	1331	c7bcb3d83c5544605bac741e625f1a441f5ba10047e204e4ce6dbd9eb7468a77
working/current	.md	Documentation/10_Development_Records/Sprint_03/Governance/ADR-0015_Project_Input_Actions_And_Context_Routing.md	1653	b665c0f33852fb4ae7d0de42fac1636487181e19c28593c1d728d412e21954e
working/current	.md	Documentation/10_Development_Records/Sprint_03/QA/S3.4_Build_Execution_Record.md	1303	bdbcbe899c627a7abf1f302f7db42ce29de1f0809c8373a454e3bc070984ab7d
working/current	.md	Documentation/10_Development_Records/Sprint_03/QA/S3_Acceptance_Matrix.md	1643	dcd3a753ec03bf462ed61fee96f551716970f05a54702fccec5ebdc271bda15b7
working/current	.md	Documentation/10_Development_Records/Sprint_03/QA/S3_QA_Execution_Record.md	2573	c36caf9cccc1d762ab9f201d7eae2f2e07ae88f1787d7ed0df6bf13305850116
working/current	.md	Documentation/10_Development_Records/Sprint_04/Governance/ADR-0016_Grid_Coordinate_And_Footprint_Foundation.md	1157	84ffb0f806ec18937785dc30f5410a0be471ea75deeb3d23021657ff513d622
working/current	.md	Documentation/10_Development_Records/Sprint_04/Governance/ADR-0017_Placement_Preview_And_Rotation.md	1125	996ccbe4400bc1971fea44b1d1e89b7a0cbe30437fa660283af16e79b8f18965
working/current	.md	Documentation/10_Development_Records/Sprint_04/Governance/ADR-0018_Atomic_Occupancy_And_Placement_Mode.md	1439	3cdafe143aedd7a015756ec256b6c92be2aaaa918c33499421589d3fb51177e1
working/current	.md	Documentation/10_Development_Records/Sprint_04/Governance/ADR-0019_Sprint_04_Final_Integration_And_Build_Gate.md	1324	f78c4971ab6969d8d3f302b93ceb5dbf30f2d553f433122bd8d24eb12fa7dff3
working/current	.md	Documentation/10_Development_Records/Sprint_04/QA/S4.4_Build_Execution_Record.md	1164	6f662c5fe7cce7a3080584798ccf51c720d4ade1dc50b1be2f8c4c107da529d
working/current	.md	Documentation/10_Development_Records/Sprint_04/QA/S4_Acceptance_Matrix.md	2209	30a86d51980351f806ef30aa6187ce90857bde474c163d47a6a3396b60522f89
working/current	.md	Documentation/10_Development_Records/Sprint_04/QA/S4_QA_Execution_Record.md	2537	0c5ec51188fd94404436c7093e01ed0a0d0d75fc466da104e777fd7494aa0603
working/current	.md	Documentation/10_Development_Records/Sprint_05/Governance/ADR-0020_Initial_Store_Shell_Dimensions_And_Entrance.md	870	af706a1741b657833a36805700e10d25e2b66264438b488c1a8983e883799e3d
working/current	.md	Documentation/10_Development_Records/Sprint_05/Governance/ADR-0021_Logical_Store_Access_Validation.md	1131	079eab3c8041ef624f17ba35976f8ac46627ed9ff192a910e1f0f4f63fee5fe7
working/current	.md	Documentation/10_Development_Records/Sprint_05/Governance/ADR-0022_Store_Placement_And_Access_Integration.md	1085	f180b5eac43281274d001667b8fc49f39febfbacfeb7fec7d686b9e07546524

Baseline	Tip o	Ruta	Bytes	SHA-256
working/current	.md	Documentation/10_Development_Records/Sprint_05/Governance/ADR-0023_Sprint_05_Final_Integration_And_Build_Gate.md	972	86eaaea910888d35b0bb89c6a30aef788d767f38b9528339487890c29844f476
working/current	.md	Documentation/10_Development_Records/Sprint_05/QA/S5.4_Build_Execution_Record.md	1347	83700192228af99fb4487bdf4cdd1332d13a194be7cd5fb197278013952f8fe1
working/current	.md	Documentation/10_Development_Records/Sprint_05/QA/S5_Acceptance_Matrix.md	2498	3c265e9509b133370788c1ea024e4de481f773b526a524f2ec880d60abe053b6
working/current	.md	Documentation/10_Development_Records/Sprint_05/QA/S5_QA_Execution_Record.md	2607	28e52cfc1189efc63cafabf899b0b94924bf2fb3edb92a43973797fcb77616d4
working/current	.md	Documentation/10_Development_Records/Sprint_06/Governance/ADR-0024_Stack_And_Unit_Capacity_Model.md	848	a621a76517cf2dce58d2f2a6444486853bceb8107e45e2ce0cead9da7b99f87
working/current	.md	Documentation/10_Development_Records/Sprint_06/Governance/ADR-0025_Atomic_Inventory_Transfers.md	873	4286c837998785523afe7b5818a07a7f09b63b8f94175a602f7133ae7c3df6ad
working/current	.md	Documentation/10_Development_Records/Sprint_06/Governance/ADR-0026_Sprint_06_Version_And_Persistence_Boundary.md	587	88e809b7e5a461f57c5d9f943ada6eeb0445f62ba3cd08440f5d2e45c48ed1f
working/current	.md	Documentation/10_Development_Records/Sprint_06/QA/S6_Acceptance_Matrix.md	3014	b55fe039c6404ff22749ff07f20dc741ee277ece3a892d9d1a6a526eb61dab8e
working/current	.md	Documentation/10_Development_Records/Sprint_06/QA/S6_QA_Execution_Record.md	2097	a7a14df3c2dc18a98c1354176d48502bafbd3309a651ea8bff3846db4c348e7c
working/current	.md	Documentation/10_Development_Records/Sprint_07/Governance/ADR-0027_Product_and_Supplier_Authoring.md	1065	86a7e5e37add2c9d6fe855fef6deda2f4ddfbaf80cdb1062dfdbec3be0dcbl3
working/current	.md	Documentation/10_Development_Records/Sprint_07/Governance/ADR-0028_Box_Based_Ordering.md	870	57af021d5e1412eadd90d34cbb62da418efa8d0bd0001d8da4c219bb9fa98f40
working/current	.md	Documentation/10_Development_Records/Sprint_07/Governance/ADR-0029_Atomic_Box_Receiving.md	1021	bff3785f3178e96118326edaf1a4134d4300f5817a838cb24bacdd7dc6884af0d
working/current	.md	Documentation/10_Development_Records/Sprint_07/QA/S7_Acceptance_Matrix.md	3351	e5517ab2f2b047aacb634c1ee7e42ea6f4b1b0f3bb6477cc7a0fb595439546b2
working/current	.md	Documentation/10_Development_Records/Sprint_07/QA/S7_QA_Execution_Record.md	2132	ed5696f59956c5341e66f5d4bd7c9ff10b1c2ae56c47f0e70f7acd4f412d62626
working/current	.md	Documentation/10_Development_Records/Sprint_08/Governance/ADR-0030_Single_Product_Display_Assignment.md	437	ff34c07064a44ae5fffb3db2f205b4ef1fe2fd330bed57702a84f53e4f428e4a5
working/current	.md	Documentation/10_Development_Records/Sprint_08/Governance/ADR-0031_Display_Capacity_and_Visible_Units.md	332	55398e2d4b0f0989db0716a124fd96d597cfe878d561d2546f8114c74009a900
working/current	.md	Documentation/10_Development_Records/Sprint_08/Governance/ADR-0032_Atomic_Manual_Restocking.md	373	3995a709fcd6235ec918570685e381fea53ef9620adb0bf10a0f25d2f0e6b970
working/current	.md	Documentation/10_Development_Records/Sprint_08/Governance/ADR-0033_Display_Authoring_and_Placement_Boundary.md	406	68b6ada32b41b33a38e6bd2e36673610f62e92d911bd9181b1cc431c25a22c77
working/current	.md	Documentation/10_Development_Records/Sprint_08/Governance/S8_Handoff_Update_Template.md	1297	752c345e77607a3d096548520f2ce082a816d2eeb0d01ea0d29ddbe2bd32013a
working/current	.md	Documentation/10_Development_Records/Sprint_08/QA/S8_Acceptance_Matrix.md	1833	514b68f2839e76b1757df9d2d457dca9701cbe8f3de4169c6e38fe7e7c1e576c
working/current	.md	Documentation/10_Development_Records/Sprint_08/QA/S8_Build_Execution_Record.md	1531	975e35974c3ac193dedc354ad39f652dccc131356a4d69ec241dddcfdd84f29c1
working/current	.md	Documentation/10_Development_Records/Sprint_08/QA/S8_Compilation_Incident_001_NUnit_Assert_Multiple.md	1971	c3f9557dbe19d9326df55919c56712a4c7b57a77e3d1044708cbe974705cb69c
working/current	.md	Documentation/10_Development_Records/Sprint_08/QA/S8_QA_Execution_Record.md	1991	6b648c6839cb4cd0f249f4318624210896b63a393cd888a3b1ac2d9f70edc61b

Baseline	Tip o	Ruta	Bytes	SHA-256
working/current	.md	Documentation/10_Development_Records/Sprint_09/Governance/ADR-0034-Deterministic_Profile_Selection_and_Spawn_Queue.md	308	b3190da9985256d7704754263838fb9845526aaf470eec40c3e6501ffa6b8d0b
working/current	.md	Documentation/10_Development_Records/Sprint_09/Governance/ADR-0035-Customer_Lifecycle_and_Patience.md	394	abae8d2eebdfc7d8b65386653fb1cbb78c351049ef3d838a6d8b0668dd1e1173
working/current	.md	Documentation/10_Development_Records/Sprint_09/Governance/ADR-0036-Technical_Navigation_Boundary.md	339	d749100129b58f4e3fd7f9da9b540340194be1dee4dce5195173796089acb4d0
working/current	.md	Documentation/10_Development_Records/Sprint_09/Governance/ADR-0037-Population_Cap_and_Technical_Representation.md	307	a03265cb88add303fe2be3c2b7982b1c183f2a3d52c5e7815b17a1a886428392
working/current	.md	Documentation/10_Development_Records/Sprint_09/Governance/S9_Handoff_Update_Template.md	501	e3d501b14918d549aab4d42cc2d289a8f1d83730600d1083109ea64ea409baae
working/current	.md	Documentation/10_Development_Records/Sprint_09/QA/S9_Acceptance_Matrix.md	727	476e8342efea9323b73b46337890b795fe2b3bc500486fd08a9e9f5dbf6706ef
working/current	.md	Documentation/10_Development_Records/Sprint_09/QA/S9_Build_Execution_Record.md	773	743edb2e5421984273fb169aab3fbb16c48d6b0ac2e177c5e83ba01b51099304
working/current	.md	Documentation/10_Development_Records/Sprint_09/QA/S9_QA_Execution_Record.md	661	ee2c2033cb369301abe3eb39bf1f0d3a71c241431b1889651e14a895437d114a
working/current	.md	Documentation/10_Development_Records/Sprint_10/Architecture/ADR-0038-ReservationBackedCart.md	176	b75acced6289b8579e95ccb0e1dbe76dcaa3fafbbd9e4c16c8bda84aaa7a37fc
working/current	.md	Documentation/10_Development_Records/Sprint_10/Architecture/ADR-0039-DeterministicShoppingSearch.md	179	f68af75f70d688595409b8c6be5689989143acb084811e26f6e21f125bae81e
working/current	.md	Documentation/10_Development_Records/Sprint_10/Architecture/ADR-0040-ReservationProvenance.md	130	fb548c17172d591576640e2454ef14319057adc48b3dbd9944374ff306f3dee5
working/current	.md	Documentation/10_Development_Records/Sprint_10/Architecture/ADR-0041-CustomerShoppingSessionBoundary.md	160	d6bf2284912c0e33992a0af84e445ad75fb6af270bb901ec0bf0f03942a11c08
working/current	.md	Documentation/10_Development_Records/Sprint_10/Governance/S10_Handoff_Update_Template.md	429	e7893c392feb7b35d3e270034b36f9da758d73673adb7e3f50f0cecea5763003a0
working/current	.md	Documentation/10_Development_Records/Sprint_10/QA/S10_Acceptance_Matrix.md	665	8a3558314d7d868e3e2ce0cd33fe62e7b1b33eddf1e70b2d6eab43c9b3600d8c
working/current	.md	Documentation/10_Development_Records/Sprint_10/QA/S10_Build_Execution_Record.md	535	adea9b7fcdab6e319f0ded3171260cc0200e2ae3ab424916a8068a7175872369
working/current	.md	Documentation/10_Development_Records/Sprint_10/QA/S10_QA_Execution_Record.md	544	c45fb3dd29d4ec4dc67ad42f496932c94cbe384096c560c08d3d1c9615c6a5fe
working/current	.md	Documentation/10_Development_Records/Sprint_11/Architecture/ADR-0042-StrictFifoCheckoutQueue.md	205	094a7bdd2663b03a04f085dd8b9e0c775081f8b64eb6319858f1ac39b086df43
working/current	.md	Documentation/10_Development_Records/Sprint_11/Architecture/ADR-0043-SingleEntryCheckoutStation.md	177	251308c9d85fe221f37152f5f94a8a811db556b3b91816c20485e7a12949d5db
working/current	.md	Documentation/10_Development_Records/Sprint_11/Architecture/ADR-0044-PreflightThenCommitCheckout.md	263	a58dfea3f71f24fd52ae8c7ab9d015aad8b31ad56c60e12162e4aff95783a864
working/current	.md	Documentation/10_Development_Records/Sprint_11/Architecture/ADR-0045-CheckoutIdempotency.md	152	dcaf08280bbc3fb2f7947d5d594d13d1d84c415dfa93e3c83148280a171d5d75
working/current	.md	Documentation/10_Development_Records/Sprint_11/Governance/S11_Handoff_Update_Template.md	422	3da63a8600ab4da7f1afe4032690763029d3551297fad0d51afbfa841687dad5
working/current	.md	Documentation/10_Development_Records/Sprint_11/QA/S11_Acceptance_Matrix.md	650	a6f876fb02398be1ee466d2f9dde9613ad9c0fa2fc0088778b791ab9a3d00e8d
working/current	.md	Documentation/10_Development_Records/Sprint_11/QA/S11_Build_Execution_Record.md	494	6e5b3ee3b6b9a1a6d2f6c90fc07df567cfd81126f6e2c06aa4c3307db206f24

Baseline	Tip o	Ruta	Bytes	SHA-256
working/current	.md	Documentation/10_Development_Records/Sprint_11/QA/S11_QA_Execution_Record.md	457	e09346a52a005447f98244392575dce8bfe0e1c24395d87ef4efa9949582222d
working/current	.md	Documentation/10_Development_Records/Sprint_12/Architecture/ADR-0046-LogicalStoreDayAggregate.md	191	4b0703008d657d9b20ef1554afc827e8126540b156009d91cfc61c6270fcc7d
working/current	.md	Documentation/10_Development_Records/Sprint_12/Architecture/ADR-0047-ClosingAdmissionGates.md	203	1bea6d3a71b2dda9f027398423d40013fed095318570d0c983c3a5ef5adb2fe
working/current	.md	Documentation/10_Development_Records/Sprint_12/Architecture/ADR-0048-DeterministicDrainAndResolution.md	201	930429172027b64eed649d79c17f55747622690c4355f303b4b1f572fe53da
working/current	.md	Documentation/10_Development_Records/Sprint_12/Architecture/ADR-0049-ClosureReadinessSnapshot.md	173	8bbcd47b5de9fe5d2fb48f877e7bc1452a95aab891c3fcae14d6b06fea819df9
working/current	.md	Documentation/10_Development_Records/Sprint_12/Architecture/ADR-0050-TechnicalDaySummary.md	146	523e7074e3ffda8a481a51a7fe1637494cf452b722c77823f268c3d7b773a1dc
working/current	.md	Documentation/10_Development_Records/Sprint_12/Governance/S12_Handoff_Update_Template.md	361	683e99d27f7af84f319fad7c98230f6dec7f9b9eeddf8b4066f750b22c230bd0
working/current	.md	Documentation/10_Development_Records/Sprint_12/QA/S12_Acceptance_Matrix.md	661	9b16fac6f71a8d05203d140a77a1b0c202a5f445556cc663208b04e5c45392b9
working/current	.md	Documentation/10_Development_Records/Sprint_12/QA/S12_Build_Execution_Record.md	431	e3531d9bc28665f33ed38c176188246308d38bd480be42c1758f848e433f69b4
working/current	.md	Documentation/10_Development_Records/Sprint_12/QA/S12_QA_Execution_Record.md	402	e53fa611bf41782ecb96b8d41010b2607f3e38223dad09898a92bdfec0f6ba9f
working/current	.md	Documentation/10_Development_Records/Sprint_13/Architecture/ADR-0051-IntegerMinorUnitMoney.md	173	73384536261c238a733a2f818844f35d55f135835e6b5c97cdd7dae6d63ca795
working/current	.md	Documentation/10_Development_Records/Sprint_13/Architecture/ADR-0052-SeparateSalePriceCatalog.md	226	1a0a34e33c5a76ff1c1fb80921aa4dbbdf223200b95c8e54bf52e95e1eece4018
working/current	.md	Documentation/10_Development_Records/Sprint_13/Architecture/ADR-0053-QuoteBeforePhysicalCheckout.md	196	5c71902b6c89d51d471338f30be7fd34014f3394ada90308b26b014956d23a
working/current	.md	Documentation/10_Development_Records/Sprint_13/Architecture/ADR-0054-IdempotentEconomyLedger.md	158	f52b16bb87159d2616add98a3c44442ddf6b5e0fe5e50fa4fea29fee33215246
working/current	.md	Documentation/10_Development_Records/Sprint_13/Architecture/ADR-0055-ClosedDayEconomicResult.md	189	d0445e9824c68ff49cdc6836a4809e56bc768dfef1f504dd17f6207db76ef0af
working/current	.md	Documentation/10_Development_Records/Sprint_13/Governance/S13_Handoff_Update_Template.md	431	7d91ce81d3567cd4f392f6d9784c7415d628ec2b65e7468ea5e1b694af743f8
working/current	.md	Documentation/10_Development_Records/Sprint_13/QA/S13_Acceptance_Matrix.md	693	e7f134005949b363e84fadbb28d740ff268adb3c42de47070fbcc968862319f2b
working/current	.md	Documentation/10_Development_Records/Sprint_13/QA/S13_Build_Execution_Record.md	493	887232c7346268536f197fc6069ad7f518735c2f51c302f0d27e6c2625cd005b
working/current	.md	Documentation/10_Development_Records/Sprint_13/QA/S13_QA_Execution_Record.md	456	a0ecdcbfd34ae6a3e96f2ff4b19602fcfdad3628da0f4647a96631aecce9bd9e0
working/current	.md	Documentation/10_Development_Records/Sprint_14/Architecture/ADR-0056-CompatibleIntegratedSaveV2.md	198	5cc539efb691df4398f23002613d08079fa8869bde84816f89a91be781ffc6fa
working/current	.md	Documentation/10_Development_Records/Sprint_14/Architecture/ADR-0057-ValidatedAtomicWrite.md	215	919cf393c0449300f85947190ba50828706c34ebfbc0a93d9aa31a1d9e021301
working/current	.md	Documentation/10_Development_Records/Sprint_14/Architecture/ADR-0058-ChecksumAndGenerationEnvelope.md	192	ee9dd8479ba19464211e32510dcb7cd3f4f4a199cc9506ac28cf7cfc3cffd0b
working/current	.md	Documentation/10_Development_Records/Sprint_14/Architecture/ADR-0059-BackupFirstRecovery.md	209	22af6a1df416b4c7fb8389f08ec428c61289bbb5984d958e0576aa2478ad03d4

Baseline	Tip o	Ruta	Bytes	SHA-256
working/current	.md	Documentation/10_Development_Records/Sprint_14/Architecture/ADR-0060-TwoPhaseRestore.md	189	4c701c8c4cfcadabb5a9002e0599765339d6028694431b79bb8a372f6a857345
working/current	.md	Documentation/10_Development_Records/Sprint_14/Governance/S14_Handoff_Update_Template.md	437	70ac9acc5473a29f44723f4e1499f027f5adbcb772ae45635ebdb035f2374bd4f
working/current	.md	Documentation/10_Development_Records/Sprint_14/QA/S14_Acceptance_Matrix.md	761	6abc7f9f6b1f199f797ede8e4610be0819c89c52ba763b09489f76e5b6d2b3e9
working/current	.md	Documentation/10_Development_Records/Sprint_14/QA/S14_Build_Execution_Record.md	490	758b82a0301e5f3de699b76365cf74c577377bcb6d88333c4b65a26e6065af33
working/current	.md	Documentation/10_Development_Records/Sprint_14/QA/S14_QA_Execution_Record.md	407	61083207815690667077272760527c8529f70916831a1bd7d75cc58125663a7f
working/current	.md	Documentation/10_Development_Records/Sprint_15/Architecture/ADR-0061-RuntimeUICompositionRoot.md	251	65c71a50143e1217e64a395d48e368db684e9051879a84fe991d293254644fd9
working/current	.md	Documentation/10_Development_Records/Sprint_15/Architecture/ADR-0062-SlotUIUsesIntegratedRepository.md	188	ddaab79fcc1ee70e54e299ef81c1b274c28de02b60cd7ca500b83f0c779a83ae
working/current	.md	Documentation/10_Development_Records/Sprint_15/Architecture/ADR-0063-ClosedDayAutosaveIdempotency.md	200	8575a38534155149713d4c711e65216ce43402c06dce60d181fbad87d806ad59
working/current	.md	Documentation/10_Development_Records/Sprint_15/Architecture/ADR-0064-TutorialProgressSidecar.md	201	d2cdde79c43ec5206b0f28cd4e4d5542210988058e9039cd6c510e41b8992b85
working/current	.md	Documentation/10_Development_Records/Sprint_15/Architecture/ADR-0065-GlobalAccessibilityPreferences.md	202	2d42a2ba223eea0dd142a1c34949bf6095b257b790b7ad36aabf0ae3fd4f30f
working/current	.md	Documentation/10_Development_Records/Sprint_15/Architecture/ADR-0066-ExclusiveUIInput.md	193	dd52812f99015a563282f87506a4b57884e6c41b4f0f9488cfb4d58944d74bbe
working/current	.md	Documentation/10_Development_Records/Sprint_15/QA/S15_Acceptance_Matrix.md	744	ef5c3d5a5d788d7a52965e858620ca76d260711f96e5132bb7938c765b6762da
working/current	.md	Documentation/10_Development_Records/Sprint_15/QA/S15_Build_Execution_Record.md	359	cb74c7605f9918b31b4f49ebbe0f63004b03886d5468bd258a3eb6f83ea9b61a
working/current	.md	Documentation/10_Development_Records/Sprint_15/QA/S15_QA_Execution_Record.md	373	317dd72193364df1dd104ef8be879fa22dbd4f8374e3e0bd454fef8f0881277d
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0067-Sprint16TwoPhaseDelivery.md	214	59f759e966b057f091473d52cf205d1a089a0917d2f8ff80342ca8ce2ca0a75
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0068-PurePhase1Sidecar.md	232	5a4b767d6daf552a48dc2b4d256b38c698da130b41314243902c4b5fbab68459
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0069-AutosaveCoordinatedCheckpoint.md	250	c4aaad9ce98b604d52894350668f7e59cc8c9d893ec3e96658a467bff0dbd44a
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0070-PlacementCompatibilityBridge.md	353	66a5b5172267c6984edd5474ba03dc2fb0da28de72c310ced74feba060f6df17
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0071-DecoupledPresentationFallbacks.md	223	f2e025be2f68b6de2388bd63a6b25549d4e6d111012bc685b034fa120ff9616c
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0072-SharedMaterialAndPrefabIds.md	223	f32e2b4b274522d03df3c1c08084d86114a19dbe15fa2b3008c217943e2ff11a
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0073-CompletedCheckoutSnapshotRecords.md	377	7d401e745e557c63167dae48cd0f4b05b1d08f5c58a2e652b4321925fc35caa7
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/QA/S16_P1_Acceptance_Matrix.md	1130	d6d2ca42287be0fced9c3d9da09df5db9729442157035a9a711e0ddb9d91337
working/current	.md	Documentation/10_Development_Records/Sprint_16/Phase_1/QA/S16_P1_QA_Execution_Record.md	443	a044ba4e7ebb4af622c74397e2a8f0ff6e59ad7e9c1bf89ddb62f5b06bcacec7

Documentos consolidados actuales consultados

- [00_Enfoque_y_Alcance.md](#) — 127401 bytes — [63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f677769d6e69ceaad21f](#)
- [01_Game_Design_Document.md](#) — 132822 bytes — [a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e781109ff146bceb2177](#)
- [02_Vertical_Slice_Specification.md](#) — 64531 bytes — [75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f](#)
- [03_Technical_Design_Document.md](#) — 101994 bytes — [66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12](#)
- [04_Modelo_de_Datos.md](#) — 102948 bytes — [d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4](#)
- [05_UX_Flow.md](#) — 56805 bytes — [e08da5ff67fb073a8b950babe325ae58ddb2e121513dc2f553acd4f2c14b867e](#)
- [06_Production_Roadmap_y_Sprint_Plan.md](#) — 61564 bytes — [d00b156c0ad1c66069278119c55f76c738a577a3e2835f770208d1a5f2faadff](#)
- [07_QA_Testing_Plan.md](#) — 79435 bytes — [bc9c05a3737e345546ef16e5390e034295ef35ad66c31647852aec81b4e7affe](#)
- [08_QA_Testing_Matrix.xlsx](#) — 185032 bytes — [233e57f01d2468fe13c136c8fcd3ee75b39bd3de6530349626ea98d4ff254214](#)
- [09_CSharp_Coding_Standards.md](#) — 95321 bytes — [3b734d6cd49f31c09c10bb4941625e143f6c634e3fb50c157f0720e73b1c2a68](#)
- [10_Unity_Project_Setup_Guide.md](#) — 78408 bytes — [31430bb0544edd9577323aa0009d6ca3df8b9fbcf6c8a150fc33df69b9f963a4](#)
- [11_Build_y_Versioning_Guide.md](#) — 85324 bytes — [38a16a0f97505c0b405509dd3c7e3d1b50e77b89fbe3f4cb1931d3e04f602215](#)
- [12_Excel_Maestro_de_Produccion.xlsx](#) — 166160 bytes — [4fe0cbfc16f96fe562a074e9a5776d0afa8d1056c3d3fbf254511dc2ed6bcefc](#)
- [13_Trazabilidad_y_Control_de_Cambios.xlsx](#) — 197997 bytes — [3303e06dd63403f64e1953c6e4d2d2efb7a0671ae25deb5c972022e9a4f4656c](#)
- [14_Project_Binder_Indice_Maestro.md](#) — 99373 bytes — [3c3672b42daf20c190ae52890743e36c9e28f078d37160396f7a380e08a14988](#)
- [15_Guia_Maestra.md](#) — 125622 bytes — [1fb7adabe86d151cb74a4b3156ca6e8a79a4fa114407c8942e28eceb0bf91e98](#)
- [16_Auditoria_Global_de_Coherencia.md](#) — 115076 bytes — [5d2807322aeadd826bf7130410347e0185f9005efaa1aa20f549299a5e4a74ce](#)
- [17_Art_Bible.md](#) — 133913 bytes — [095c6cc42223a972c7faa68435595fc7c22f839fab62b2038371a0ef6c14239](#)
- [18_Audio_Bible.md](#) — 144525 bytes — [c2eabe93a7c26f70e1d4d77fb0ddcaa97e2cdd1e70247e637b9db5bd2f07ce92](#)
- [19_UI_Style_Guide.md](#) — 149800 bytes — [b34ca4db1eba7a536466b6a9a105d8d92396ade2108aa7dd99fa7d5d4eaebc1a](#)
- [20_Economy_and_Balance_Specification.md](#) — 154547 bytes — [908c70772ed35eecd05c3036c6901523a4caf2c3e708ca8f049c2a160ef2aac](#)

- 21_Initial_Content_Catalog.xlsx — 174383 bytes — 9047299f60cfbd368fc4a2c8bee8d0ff4132dc5642363a9a67d1523559eb9ed3
- 22_Localization_Plan.md — 157323 bytes — 184a53b2813bc1327a6913e6e88ea4c81142da1ee4ddef88f75467c784f1ff0
- 23_Legal_Credits_and_Licenses_Register.xlsx — 465135 bytes — f543b3b5de8900f405910fac5d72697045f8c4969d98fce4c1cf4f12e532b2f1
- 24_Steam_Publishing_Plan.md — 171037 bytes — f087e24d3a18f8c4a6651728d5f70d1df804042587e206fdeac2e7e637a87646
- 25_Post_Launch_and_Live_Operations_Plan.md — 327218 bytes — 158e1e43514c5d5d6c365f6cd1a73929f54c4a48c5cc460b593530ae5e49bac9
- 26_Privacy_Data_and_Telemetry_Plan.md — 321182 bytes — 284abfe537793c2ec8b0e5cbac484cd9395567c4434aef7f5ad5ee7005e95f36

172. Historial de cambios

Fecha	Versión	Cambio	Autoridad
2026-07-01	v1.0-consolidated	Creación del plan autónomo a partir de baselines v0.3-v0.6, ADR, código/configuración real y documentos 00-26.	VRM Games
2026-07-01	v1.1-verified	Incorporación de NIST SP 800-61 Rev. 3, estado técnico real, package inventory, Unity settings, secret/network scan, incident runbooks y gates.	VRM Games

Próxima revisión obligatoria

- antes de onboarding Steamworks;
- antes de external playtest o beta;
- antes de H6/release candidate;
- al añadir backend, Cloud, telemetría, crash reporting, Workshop o colaboradores;
- tras cualquier SEC-0, SEC-1 o near miss relevante.

Este historial registra cambios del documento. La ejecución de controles y el cierre de riesgos se registran en Producción, Trazabilidad, H6 y Risk Register.

Fin del documento.

Fuente: 27_Security_and_Incident_Response_Plan.md · SHA-256: `f925bfbf3df2000323c12a4486480d06c8ab8181684e5922168b8bdf89d05f92`

Diseño PDF: paleta VRM Games definida en la documentación de Cartridge & Cloud.